Zip Code Wilmington's Programming in Java

Kristofer Younger

Version 2.1, 2025-06-01

Table of Contents

Colophon . . . 1
Preface . . . 2
About this book . . . 3
- Java: Pretty Easy to Understand . . . 3
- Coding *The Hard Way*. . . . 3
- Dedication to the mission . . . 5

1. Output - System.out.println() . . . 7
2. Comments . . . 10
   - 2.1. Chapter Exercises . . . 11
3. BUT WAIT!!! . . . 13
   - 3.1. Two kinds of Numbers in Java . . . 13
4. Statements and Expressions . . . 16
   - 4.1. Expressions . . . 16
   - 4.2. Statements . . . 17
   - 4.3. Multi-line Statements . . . 17
   - 4.4. Block Statement . . . 18
5. Variables and Data Types . . . 20
   - 5.1. Variables . . . 20
   - 5.2. Constants . . . 23
   - 5.3. Data Types . . . 24
   - 5.4. Chapter Exercises . . . 26
6. Arithmetic Operators . . . 28
   - 6.1. Basics . . . 28
   - 6.2. Division and Remainder . . . 30
   - 6.3. Order is Important . . . 31
   - 6.4. Java Math Class . . . 34
   - 6.5. Chapter Exercises . . . 35
7. Algebraic Equations . . . 38
   - 7.1. *Trigonometry* . . . 39

7.2. Chapter Exercises . . . 40
8. Simple Calculation Programs . . . 43
8.1. How far can we go in the car? . . . 43
8.2. The Cost of a "Free" Cat . . . 44
8.3. You Used Too Much Data! . . . 45
9. Boolean Expressions . . . 48
9.1. Comparison Operators . . . 49
9.2. Logical Operators . . . 50
9.3. Chapter Exercises . . . 52
10. Strings . . . 55
10.1. What is a String? . . . 55
10.2. Declaring a string . . . 57
10.3. String Properties . . . 57
10.4. Accessing Characters in a String . . . 57
10.5. String Concatenation (Joining strings) . . . 58
10.6. SubStrings . . . 58
10.7. Summary of substring methods . . . 60
10.8. Chapter Exercises . . . 60
11. Arrays . . . 63
11.1. Declaring Arrays . . . 64
11.2. Accessing elements of an Array . . . 65
11.3. Get the size of an Array . . . 65
11.4. Get the last element of an Array . . . 65
11.5. Change an element of an Array . . . 66
11.6. Chapter Exercises . . . 66
12. Changing the Control Flow . . . 69
13. Conditional Statements . . . 70
13.1. If statement . . . 70
13.2. Chapter Exercises . . . 72
13.3. Switch Statement . . . 75
14. Loops . . . 77

14.1. While Loop . . . 77
14.2. Do..While Loop . . . 79
14.3. For Loop . . . 79
14.4. Chapter Exercises . . . 81
14.5. Break Statement . . . 84
14.6. Continue Statement . . . 84
15. Code Patterns . . . 86
15.1. Simple Patterns . . . 86
15.2. Loop Patterns . . . 87
15.3. Array Patterns . . . 88
16. Classes . . . 91
17. Methods . . . 92
17.1. method Literal . . . 92
17.2. Creating a method . . . 93
17.3. Invoking methods . . . 93
17.4. Anonymous methods . . . 94
17.5. method Return . . . 95
17.6. method Parameters . . . 96
18. Return statement . . . 99
18.1. Chapter Exercises . . . 100
19. Objects . . . 102
19.1. Object Creation . . . 103
19.2. Property Retrieval . . . 105
19.3. Property Update . . . 105
19.4. Object Reference . . . 105
19.5. Object Functions . . . 106
19.6. Follow Ons . . . 108
Appendix A: Kris' Special Class . . . 110
A.1. The SPECIAL ZipCode java template! . . . 110
Appendix B: Advanced Ideas . . . 111
B.1. Simplifying Loops . . . 112

B.2. Simplifying Conditionals 116
B.3. Lambdas (or Arrow Functions) 118
B.4. Polymorphism and K.I.S.S. 119
Appendix C: Mars Lander 123
Appendix D: Installing Java JDK 128
D.1. Checking Your Current Java Installation 128
D.2. Installing Java JDK 129
D.3. Verifying Your Installation 131
D.4. Troubleshooting Common Issues 131
D.5. Recommended Java Versions 133
D.6. Getting Help 133
Appendix E: Additional Java Resources 134

Colophon

Zip Code Wilmington's Programming in Java by Kristofer Younger

Copyright © 2020-2025 by Zip Code Wilmington. All Rights Reserved.

Published in the U.S.A.

November 2024: First Edition

While the publisher and author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions in this work is at your own risk. If any code samples or other information this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

Preface

I'd like to thank the instructors of ZipCodeWilmington for inspiring my series of programming books: Chris Nobles, Roberto DeDeus, Tyrell Hoxter, L. Dolio Durant, and Mikaila Akeredolu. Without them, this book would have remained in "maybe some day" category. My thanks to Dan Stabb and Roberto, who made corrections to the text. I hope you get a chance to code someday, Dan! I especially wish to thank Mikaila: without his brilliant prep session slides as the starting point for this book, I would never have thought a small book on the basic fundamentals of programming would be possible or even useful.

Zip Code Wilmington is a non-profit coding boot-camp in Delaware for people who wish to change their lives by becoming proficient at coding. Find out more about us at https://zipcodewilmington.com

About this book

This book's aim is to provide you with the most basic of fundamentals regarding Java, the programming language we have been teaching at Zip Code Wilmington for more than 8 years. To someone who has spent some time with programming languages, this might be just a breezy intro to Java. If you have almost any serious coding experience, this book is probably too elementary for you. You might, however, find the ideas in the Appendices interesting. There I've added some material that have a few advanced ideas in them, plus there is a full code listing of a mars lander simulation.

Java: Pretty Easy to Understand

Java is a fairly easy programming language to learn, and we're going to use it in this book to describe the basic fundamentals of coding. Its form is modern and it is widely used. It can be used for many purposes, from web applications to back-end server processing. We're going to use it for learning simple, programming-in-the-small, object-orientation, methods and concepts; but make no mistake - Java is world-class language capable of amazing things.

Coding *The Hard Way*.

Zed A. Shaw is a popular author of several books where he describes learning a programming language *The Hard Way*. Zed suggests, and we at Zip Code agree with him whole-heartedly, that the best, most impactful, highest return for your investment when learning to code, is **type the code using your own fingers** [1]

That's right. Whether you are a "visual learner", a "video learner", or someone who can read textbooks like novels (are there any more of these out there?), the best way to learn to code is **to code** and **to code by typing out the code with your own fingers**. This

means you DO NOT do a lot of copy and paste of code blocks; you really put in the work, making your brain better wired to code by **coding with your own typing of the code.**

You're here, reading this, because you're thinking (or maybe you know) that you want to become a coder. It's pretty straight-forward.

You may have heard a friend wistfully dream of making a career at writing. "Oh," they say, "I wish I had time to write a great novel, I want to be a writer someday".

So you can ask them: Did you write *today*? *How many words?* And the excuses flow: "Oh, I have to pick up the kids" "Ran out of time, I'm so busy at work." "I had to cut the grass" and so on. Well, I'm here to tell you that all the excuses in the world don't stop a real writer from writing. They just sit down and do it. As often as they can, sometimes even when they can't (or shouldn't).

Coding, like writing, isn't something you can do when all your other chores, obligations, and entertainments are done. If you're serious about learning coding, you must make time for coding.

Watching hours of YouTube videos *will not* make you a coder.

Reading dozens of blog posts, Medium articles, and books *will not* make you a coder.

Following along with endless step-by-step tutorials *will not* make you a coder.

The only way you're going to learn to code is by doing it. Trying to solve a problem. Making mistakes, fixing them, learning from what worked and what didn't at the keyboard.

Many have heard my often-repeated admonition: **If you coded today, you're a coder. If not, you're not a coder.** It really is as simple as that.

Dedication to the mission

I happen to be among those who feel anyone can learn to code. It's a 21st century superpower. When you code, you can change the world. Being proficient at coding can be a life-changing skill that impacts your life, your family's life and your future forever. Time and time again, I've seen that the ability to learn to code is evenly distributed across the population, but the *opportunity to learn to code is not*. So, we run Zip Code to give people a shot at learning a 21st century superpower, no matter where you come from.

And fortune favors the prepared. Some day, you may be working at a great company, making a decent living, working with professionals in a great technical job. Your friends may say "You are so lucky!"

And you will think: **Nope. It wasn't luck.** You'll know that truly. You got there by preparing yourself to get there, and by working to get there, working *very hard*. Ain't no luck involved, just hard work. You make your own luck by working hard.

As many know, getting a spot in a Zip Code cohort is a hard thing to do. Many try but only a few manage it. I often get asked "what can I do to prepare to get into Zip Code?"

The best way is to start solving coding problems on sites like https://hackerrank.com - HackerRank (among others) has many programming assignments, from extremely simple to very advanced. You login, and just do exercise after exercise, relieving you of one of the hardest of coding frustrations, that of trying to figure out **what** to code. Solving programming assignments is a good way to start to cultivate a coding mindset. Such a mindset is based on your ability to pay very close attention to detail, a desire to continually learn, and being able to stay focused on problem solving even if it takes a lot of grit and dedication.

Spending even 20 minutes a day, making progress on a programming task can make all the difference. Day after day your skills will grow, and before long you'll look back on the early things you did and be astonished as to how simple the assignments were. You may even experience embarrassment at remembering how hard these simple exercises seemed at the time you did them. (It's okay, we've all felt it. It's part of the gig.)

Working on code every day makes you a coder. And coding everyday will help with your ability to eventually score high enough on the Zip Code admissions assessment that you get asked to group and potentially final interviews. And then, well, then you get to learn Java or Python and work yourself to exhaustion doing so. Lots and lots more hours.

Why?

You do that hard work, you put in those hours, you create lots of great code, you'll make your own luck, and someone will be impressed and they will offer you a job. And that is the point, right? A job, doing what you love, coding. Right? RIGHT?

You're Welcome,

-Kristofer

Ready?

Okay, let's go.

[1] check out his terrific work: https://learncodethehardway.org

Chapter 1. Output - System.out.println()

Let's start with a really simple program. Perhaps the simplest Java program is:

```
System.out.println("Hello, World!");
```

This program just prints "Hello, World!".

System.out.println, in this case, means *outputting* or *printing* the "Hello, World!" somewhere. That somewhere is probably the terminal where the program was run.

We will use `System.out.println` to do a lot in the coming pages. It's a simple statement, one that in other languages might be `print` or `write` or `puts` (and why we all cannot just agree on one way to do this, well, beats me. Java's is `System.out.println`)

Here's your second Java program:

```
double milesPerHour = 55.0;
System.out.println("Car's Speed: " + milesPerHour);
```

Now, how do we `run` this program? Use jshell!

jshell is Java's built-in interactive tool that comes with Java 9 and later. It lets you type Java code and see results immediately, without having to create full programs with classes and main methods.

To start jshell, open your terminal or command prompt and type:

```
jshell
```

You'll see a prompt like this:

```
|  Welcome to JShell -- Version 17.0.2
|  For an introduction type: /help intro

jshell>
```

*(And if you get something like **jshell: command not found**, see the appendix at the back of the book on how to install what you need.) --- (I'll wait. Go ahead, come back here when it's installed.)*

Now you can type simple Java statements directly! For example:

```
jshell> double milesPerHour = 55.0
milesPerHour ==> 55.0

jshell> System.out.println("Car's Speed: " + milesPerHour)
Car's Speed: 55.0
```

Throughout this book, code examples are designed to work directly in jshell. You can type these statements one by one and see immediate results. For example:

```
jshell> double milesPerHour = 55.0;
jshell> System.out.println("Car's Speed: " + milesPerHour);
```

jshell is perfect for learning Java because you can experiment immediately without the overhead of creating full programs. You can: - Test individual statements - Try different values - See results instantly - Fix mistakes quickly

To exit jshell when you're done, type `/exit` or press Ctrl+D.

Cool, huh? The ability to communicate with you is one of Java's most fundamental capabilities. And you've run Java code interactively. Congratulations, you're a coder. (Well, at least for today you are.)

Chapter 2. Comments

While you're not thinking about the long term, or about large Java programs, there is a powerful thing in Java that helps with tracking comments and notes about the code.

In your program, you can write stuff that Java will ignore, it's just there for you (or readers of your code). We use two slashes to start a comment, and the comment goes to the end of the line. Java will ignore anything on a line after two forward slashes. "//"

```
// this is a comment. it might describe something in the code.
int x = 53 - 11;

int y = x * 4; // this is also a comment.
```

Often, you'll see something like this in this book.

```
double flourAmount = 3.5;
System.out.println(flourAmount); // -> 3.5
```

That comment at the end of the System.out.println line is showing what you can expect to see in the output. Here it would be "3.5" printed by itself. Try it in jshell:

```
jshell> double flourAmount = 3.5;
jshell> System.out.println(flourAmount);
3.5
```

We can also add useful stuff to the .log call.

```
double flourAmount = 3.5;
System.out.println("We need " + flourAmount + " cups of flour.");
```

See how Java types it all out as a useful phrase? That proves to be very handy in a million-plus (or more) cases.

Comments can be used to explain tricky bits of code, or describe what you should see in output. Comments are your friend.

2.1. Chapter Exercises

Note on exercises: So yeah, I want you to learn Java. And you claim you want to learn Java. So, then, about the these exercises; remember when I said learning the hard way is best? (the idea that by typing code in with your own two-little-fingers (oh wait, what? you know how to type? You'll go far in this gig!!)) Yeah, so here's the deal.

You should NOT copypasta these exercises into jshell. You should t-y-p-e them. Yes, that's right, t-y-p-e them in with your fingers. You will practice what code looks like and how it goes together if you do enough of it. And that word, "copypasta", well look it up.

Try these exercises in jshell to practice System.out.println and basic Java syntax:

Exercise 1: Your First Output

Print your name and age using System.out.println: Replace [Your Name] and [Your Age] with your actual information.

```
System.out.println("My name is [Your Name]");
System.out.println("I am [Your Age] years old");
```

Exercise 2: Using Variables and Concatenation

Create variables for your favorite food and hobby, then print them:

```
String favFood = "pizza";
String hobby = "reading";
System.out.println("I love eating " + favFood);
System.out.println("My favorite hobby is " + hobby);
```

Exercise 3: Numbers and Math

Create variables for two numbers and print their sum:

```
int num1 = 15;
int num2 = 27;
System.out.println("Sum of " + num1 + " & " + num2 + " is " +
(num1 + num2));
```

Exercise 4: Multiple Data Types

Practice with different data types:

```
String name = "Java";
int version = 21;
double pi = 3.14159;
boolean isAwesome = true;

System.out.println("Language: " + name);
System.out.println("Version: " + version);
System.out.println("Pi: " + pi);
System.out.println("Is awesome: " + isAwesome);
```

Chapter 3. BUT WAIT!!!

What does `int` and `double` signify? Well, see how they proceed another word?

```
int loavesToMake = 2;
double flourAmount = 3.5;
//  ^^^^^^ ^^^^^^^^^^^
System.out.println("We need " + flourAmount
  + " cups of flour to make "+loavesToMake+" loaves.");
```

It is our way of telling Java what `flourAmount` is. We want `flourAmount` to be place holder for the number `3.5`, because makes the code easier to read and understand. See much more on this, up ahead, in `variables`.

3.1. Two kinds of Numbers in Java

There are times when I cannot believe I have to make this point. It just seems like, "well, if Java is so smart why do I have to care about this."

What am I complaining about?

Well, to make a strange distinction between **two** types of numbers, Java has a bunch different types of numbers.

In Java, we have to decide what **type** of number we are going use with a variables like *flourAmount* and *loavesToMake*.

If you're just going to *count* things (like say the number of birthdays a person has had), we will use *integers* for that. *integers* in Java, are called **int**. A `int` means a number, but just an integer like 0, 1, 2, 42, 99, etc. It even means negative numbers -5 or -123487262. The math teacher might say that *integers* are *whole numbers*.

But a **lot** of things need a lot more flexibility. The value of your debit account could be $105.55, which is a number *between* 105 and 106. The amount of fuel in your car's gas tank isn't 1 or 6 gallons, it's probably something like 7.42 gallons, and changes by small amounts as you drive.

So Java has another **type** of number, a **floating point number** and in these early days in this book, we will use **double** as the kind of number we use when we need a number that might not be a *whole number*, but rather *real numbers*. A **double** can have decimal point and some number after, so I can write `3.5` or `42.0`. Or even more complicated numbers like:

```
3.14159
1000000.000001
7.5
55.0
14.0506
0.00678
```

We will use throughout this book, two *types* of numbers. **int**s which are whole numbers like

```
2
0
-16
4560072615728
```

And there are **double**s which are real numbers like

```
1.5
0.0
-7.005
822.98676253
```

And it turns out Java has a few more **types** of numbers, more

than just **int** and **double**. I'll list them, but you'll need to go to a larger Java book or website to understand how they are different from *int* and *double*. They are four more **types** of numbers: **long**, **short**, **byte**, and **float**.

Chapter 4. Statements and Expressions

In Java, there are parts of a program and different parts have different names. Two of the most basic (and fundamental) are **statements** and **expressions**.

4.1. Expressions

An **expression** is something that needs to be *computed* to find out the answer. Here are a few simple ones.

```
2 + 2 * 65536

speed > 55.0

regularPrice * (1.0 - salePercentOff)
```

Each of these lines is something we'd like Java to **compute** for us. That computation is often referred to as "evaluation" or "evaluate the expression" to get to the answer. There are two kinds of expressions in Java, *arithmetic expressions* and *boolean expressions*.

Arithmetic expressions are, as their name implies, something that require arithmetic to get the answer. An expression like "5 + 8 - 3" gets *evaluated* to 10. So an arithmetic expression will end up being a *number*.

Boolean expressions result in either a True or a False value. Example: "maxSpeed > 500.0" - this is either true or false depending on the value of maxSpeed. Or "homeTeamScore > visitorTeamScore" which will be true is and only if the Home team's score is greater than the Visitor team's score. Both *scores* are numbers, the result of the **greater than** makes the result of

the epxression a *boolean expression* (either **true** or **false**).

4.2. Statements

A **statement** is just a line of Java. It ends with a ';' (semi-colon).

```
// at the Grocery
double salesTaxRate = 0.06;
double totalGroceries = 38.99;
double salesTax = totalGroceries * salesTaxRate;
double chargeToCard = totalGroceries + salesTax;
```

And this is what a Java program looks like. It's just a list of statements, one after the other, that get computed from the top down.

Some of the statements have expressions in them (like totalGroceries * salesTaxRate), while some are just simple **assignment** statements (like totalGroceries = 38.99, where we assign the variable 'totalGroceries' the value 38.99). Don't panic. These are just some simple examples of Java to give you a feel for it. We'll go thru each of these kinds of things slowly in sections ahead.

4.3. Multi-line Statements

In this book, you may see that the code used in examples is longer than can fit on one line in the code boxes. Well, Java doesn't care. That's why it has **semi-colons** ';' at the end of the statements. So to be clear, a statement with long variable names is the same as one with a short name.

```
k = h * kph - (rest / 60);

kilometersCycled = numberOfHoursPedalled * kilometersPerHour -
```

```
(totalMinutesOfRest / 60);
```

When you come across code that goes onto multiple lines, do like Java does, read until you find the ';'. It's like a period in an English sentence.

4.4. Block Statement

Very often in Java, we will see a **block** of statements. It is a list of statements inside of a pair of curly-braces "{ }". It acts like a container to make clear what statements are included in the block.

```
if (magePower > 120.0) {
    maxMagic = 500.0;
    lifeSpan = 800.0;
    maxWeapons = magePower / maxPowerPerWeapon;
}

// some more code
```

See those curly-braces? They start and stop the *block*, and contain the statements within. You can also see how the code is indented, but the real key are those braces. You'll see lots of blocks when you're looking at Java code.

And, it's a bad idea, but this code:

```
if (wizardStrength > 120.0) { maxMagic = Wizard.maxpower();
lifeSpan = Wizard.maxlife(); maxWeapons = wizardStrength *
numberOfWands; }
```

is identical to this:

```
if (wizardStrength > 120.0) {
```

```
    maxMagic = Wizard.maxpower();
    lifeSpan = Wizard.maxlife();
    maxWeapons = wizardStrength * numberOfWands;
}
```

But the SECOND example is formatted much cleaner, making it more readable and therefore easier to understand. So I encourage you to your code easy to read by keeping it tidy.

Chapter 5. Variables and Data Types

5.1. Variables

In Java, variables are containers used to store data while the program is running. Variables can be named just about anything, and often are named things that make you think about what you store there.

Think of them as little boxes you can put data into, to keep it there and not lose it. If you wanted to track a number, an age of someone, you can create a variable to keep it in.

```
int age = 3;
```

Now, if we wanted to indicate that someone had a birthday, we might add 1 to the variable.

```
age = age + 1;
```

At this point, the *age* variable as 4 in it.

There are some rules about variable names.

- All Variables must be named.
- Names can contain letters, digits, underscores, and dollar signs
- Names must begin with a letter
- Names can also begin with $ and _ but are often used in special cases
- Names are case sensitive (y and Y are different variables)

- Reserved words (like Java keywords) cannot be used as names
- All variable names must be unique (no two alike)

So this means the following are fine for variable names in Java:

```
x
area
height
width
currentScore
playerOne
playerTwo
$sumOfCredits
_lastPlay
isPlayerAlive
```

And uppercase and lowercase letters are different. So each of these are DIFFERENT variables even though they are based on the same word(s).

```
currentSpeed
current_speed
currentspeed
CURRENT_SPEED
```

So be careful with UPPERCASE and lowercase letters in variable names.

5.1.1. Declaring a Variable

```
int x;
```

This creates a variable **x** that can hold an integer (int) primitive type. But it has NO value assigned to it, so if we...

```
System.out.println(x);  // -> undefined, although it will
probably print zero.
```

If we were to declare a variable named 'cats' and assign it the value 3 (because we have 3 cats?):

```
int cats = 3;
System.out.println(cats);  // -> 3
```

5.1.2. Assign a value to a variable

```
int age = 19;
String name = "James";
System.out.println(name + " is " + age + " years old"); // ->
James is 19 years old

age = 21;
name = "Gina";
System.out.println(name + " is " + age + " years old"); // ->
Gina is 21 years old
```

NOTICE how the **first time** we use *age* and *name* we must *declare* those variables. But the *second time* (and any use thereafter) we don't have to put *int* or *String* in front of the name.

This is how we make sure we don't use variables we have not declared (which in Java is a *syntax* error).

5.1.3. Reassigning a value to a variable

```
String x = "five";
System.out.println(x); // -> five
x = "nineteen";
System.out.println(x); // -> nineteen
```

Notice how we don't use "String" again, when we assign "nineteen" to x. We can assign a variable as many times as we might need to.

```
int age = 3;
// have a birthday
age = age + 1;
// have another birthday
age = age + 1;
System.out.println(age); // -> 5
```

Notice here how age's current value is used, added one to it, and re-assigned *back into the variable *age**.

Java, once you declare a variable as an *int*

```
int height = 62.0; // inches maybe?
System.out.println(height); // -> 62

height = "very tall"; // Java WILL NOT LET you do this.
System.out.println(height);
```

Java demands that once you tell it height is a int, you cannot change it to a String. You might think, but hey, *I'm the programmer*, but in fact, Java is protecting you from a common logic error. And in doing so, is shows what we mean by Java being a **strongly-typed language**. Once you declare something to be of a given type, you cannot change the type while the program is running.

5.2. Constants

Constants are like let variables but they contain values that do NOT change such as a person's date of birth. Convention is to capitalize constant variable names.

You make a double or int or String a constant by marking it **final**.

```
final String DATE_OF_BIRTH = "04-02-2005";
DATE_OF_BIRTH = "04-10-2005"; // <-error

final double SPEED_OF_LIGHT = 670_616_629.0; // miles per hour

SPEED_OF_LIGHT = 700_000_000.0; // ERROR, cannot changed things
marked final
```

An attempt to re-assign a value to a constant (something marked with *final*) will fail.

5.3. Data Types

Java can keep track of a number of different kinds of data, and these are known as "primitive data types". There are 5 of them.

- **Number** - there are two kinds of numbers: integers and floating point
  - **Integers** (or **int**) are like 0, -4, 5, 6, 1234
  - **Floating Point** (or **double**) are numbers where the decimals matter like 0.005, 1.7, 3.14159, -1600300.4329
- **String** - an array of characters -
  - like 'text' or "Hello, World!"
- **Boolean** - is either **true** or **false**
  - often used to decide things like isPlayer(1).alive() [true or false?]

It is common for a computer language to want to know if data is a bunch numbers or text. Tracking what **type** a piece of data is is very important. And it is the programmer's job to make sure all the data get handled in the right ways.

So Java has a few fundamental **data types** that it can handle. And we will cover each one in turn.

Exercise: Create Variables for Primitive Data Types

Create variables for each primitive data type:

- double
- int
- string
- boolean

Store a value in each.

```
// Here are some samples.

// integer
int x = 0;

// boolean
boolean playerOneAlive = true;

// float
double currentSpeed = 55.0;

// string
String playerOneName = "Rocco";

// constant integer

final int maxPainScore = 150000;
```

Now, you try it. Write down a variable name and assign a normal value to it.

```
// Here are some samples.

// integer
int applesPerBushel =     ;

// boolean
boolean isEngineRunning =          ;
```

```
// float
double myGradePointAverage =       ;

// string
String myDogName =         ;

// constant integer

final int floorsInMyBuilding =         ;
```

5.4. Chapter Exercises

Try these exercises in jshell to practice working with variables:

Exercise 1: Basic Variable Declaration and Assignment

Create variables for a student's information:

```
String studentName = "Alex";
int age = 20;
double gpa = 3.85;
boolean isEnrolled = true;

System.out.println("Student: " + studentName);
System.out.println("Age: " + age);
System.out.println("GPA: " + gpa);
System.out.println("Enrolled: " + isEnrolled);
```

Exercise 2: Variable Reassignment

Start with a score and update it:

```
int score = 100;
System.out.println("Initial score: " + score);

score = score - 15;  // Lost points
System.out.println("After penalty: " + score);
```

```
score = score + 25;  // Bonus points
System.out.println("After bonus: " + score);
```

Exercise 3: Working with Constants

Create constants for a pizza order:

```
final double PIZZA_PRICE = 12.99;
final double TAX_RATE = 0.08;
final String RESTAURANT_NAME = "Tony's Pizza";

int quantity = 3;
double subtotal = PIZZA_PRICE * quantity;
double tax = subtotal * TAX_RATE;
double total = subtotal + tax;

System.out.println("Order from " + RESTAURANT_NAME);
System.out.println("Quantity: " + quantity);
System.out.println("Subtotal: $" + subtotal);
System.out.println("Tax: $" + tax);
System.out.println("Total: $" + total);
```

Exercise 4: Variable Naming Practice

Fix these poorly named variables by creating better ones:

```
// Poor names:
int x = 25;
String s = "John";
double d = 5.75;

// Better names:
int studentAge = 25;
String firstName = "John";
double hourlyWage = 5.75;

System.out.println(firstName + " is " + studentAge + " years
old");
System.out.println("Hourly wage: $" + hourlyWage);
```

Chapter 6. Arithmetic Operators

Java can do math. *(and that should not surpirise you!)* And many early programming exercises you will come across deal with doing fairly easy math. There are ways to do lots of useful things with numbers.

Now, remember, there are two basic kinds of numbers, integers and floating-point. And these operators work on both kinds.

6.1. Basics

Operator	Name	Description
+	Addition	Add two values
-	Subtraction	Subtract one from another
*	Multiplication	Multiply 2 values
/	Division	Divide 2 numbers
%	Modulus	Remainder after division
++	Increment	Increase value by 1
- -	Decrement	Decrease value by 1

Say we needed to multiply two numbers. Maybe 2 times 3. Now we could easily write a program that printed that result.

```
System.out.println(2 * 3);
```

And that will print 6 on the console. But maybe we'd like to make it a little more complete.

```
int a = 2;
int b = 3;
// Multiply a times b
int answer = a * b;
System.out.println(answer); // -> 6
```

Here we have set up two variables, a and b, for our *operands* and a final *answer* variable. But, it's pretty much the same as the more simple example above.

Using this as a model, how would you write programs to solve these problems?

Exercises:

- Exercise 1: Subtract A from B and print the result
- Exercise 2: Divide A by B and print the result
- Exercise 3: Use an operator to increase A by 1 and print result
- Exercise 4: Find remainder of A of b

```
int a = 9;
int b = 3;

int L1 = b - a;
int L2 = a / b;
int L3 = a + 1;
//or using increment
L3 = a++;
int L4 = a % b;
System.out.println(L1); // -> -6
System.out.println(L2); // -> 3
System.out.println(L3); // -> 10
System.out.println(L4); // -> 0
L4 = 10 % b;
System.out.println(L4); // now -> 1
```

6.2. Division and Remainder

We know that we can do regular division. If have a simple program like this, we know what to expect:

```
int a = 6 / 3; // -> 2
int a = 12 / 3; // -> 4
int a = 15 / 3; // -> 5
int a = 10 / 4; // -> 2 (what??)
```

When we are dividing *int_s, we always get an _int* as a result. So, somewhat confusingly, when we evaluate `10 / 4` we get `2`. (and remember in school when the teacher would say, "10 / 4 is 2 with a remainder of 2"?? yes, remainders aare still with us.)

But sometimes, we have a need to know what the *remainder* of an integer division is. The remainder operator % (the percent sign), despite its appearance, is not related to percents.

The result of a % b is the remainder of the integer division of a by b. So as above, if we evaluate `10 % 4` we will get the remainder of `2`.

```
System.out.println( 5 % 2 ); // 1, a remainder of 5 divided by 2
System.out.println( 8 % 3 ); // 2, a remainder of 8 divided by 3
```

Now what's this about '%' (the remainder) operator?

```
int a = 3;
int b = 2;
// Modulus (Remainder)
int answer = a % b;
System.out.println(answer); // -> 1
```

```
int a = 19;
```

```
int b = 4;
// Remainder
int answer = a % b;
System.out.println(answer); // -> 3
```

```
int a = 4;
int b = 4;
// Remainder

for (a = 4; a < 13; a++) {
    System.out.println(a + " " + (a % b)); // would produce
}
    // output of
    // 4 0
    // 5 1
    // 6 2
    // 7 3
    // 8 0
    // 9 1
    // 10 2
    // 11 3
    // 12 0
  }
}
```

6.3. Order is Important

A strange thing about these operators is the order in which they are evaluated. Let's take a look at this expression.

$$6 \times 2 + 30$$

We can do this one of two ways:

- Say we like to do multiplication *(I know, who is that?)*
  - we would then do the "6 times 2" part first, giving us 12.
  - then we'd add the 30 to 12 giving us 42 [1]

- But say we don't like multiplication, and want to save it for later…
  - we would add 2+30 first, giving us 32
  - and then we multiply it by 6, and, whoa, we get 192!

Wait! Which is right? How can the answers be so different, depending on the order we do the math in? Well, this shows us that the Order of Operations is important. And people have decided upon that order so that this kind of confusion goes away.

Basically, multiply and divide are higher priority than add and subtract. And exponents (powers) are highest priority of all.

There is a simple way to remember this.

6.3.1. P.E.M.D.A.S

Use this phrase to memorize the default order of operations in Java.

Please Excuse My Dear Aunt Sally

- Parenthesis ( )
- Exponents 2^3
- Multiplication * and Division /
- Addition + and Subtraction -

Divide and Multiply rank equally (and go left to right) So, if we have "6 * 3 / 2", we would multiply first and then divide. "6 * 3 / 2" is 9

Add and Subtract rank equally (and go left to right) So if we have "9 - 6 + 5", we subtract first

> and then add. "9 - 6 + 5" is 8

Problem: Order of Operations

30 + 6 × 2 How should this be solved?

Right way to solve 30 + 6 × 2 is first multiply, 6 × 2 = 12, then add 30 + 12 = 42

This is because the multiplication is *higher priority* than the addition, *even though the addition is before the multiplication* in the expression. Let's check it in Java:

```
int result = 30 + 6 * 2;
System.out.println(result);
```

This gives us 42.

Now there is another way to force Java to do things "out of order" with parenthesis.

Problem: Using Parentheses

(30 + 6) × 2

What happens now?

```
int result = (30 + 6) * 2;
System.out.println(result);
```

What's going to happen? Will the answer be 42 or 72? Try it in jshell and find out:

```
jshell> int result = (30 + 6) * 2;
jshell> System.out.println(result);
```

```
72
```

6.4. Java Math Class

There is a useful thing in Java called the Math class which allows you to perform mathematical tasks on numbers.

- Math.PI; - returns 3.141592653589793
- Math.round(4.7); // returns 5
- Math.round(4.4); // returns 4
- Math.pow(x, y) - the value of x to the power of y - x^y
- Math.pow(8, 2); // returns 64
- Math.sqrt(x) - returns the square root of x
- Math.sqrt(64); // returns 8

What does "returns" mean?

When we ask a 'function' like sqrt to do some work for us, we have to code something like:

```
double squareRootTwo = Math.sqrt(2.0);
System.out.println(squareRootTwo);
```

We will get "1.4142135623730951" in the output. That number (squareRootTwo) is the square root of 2, and it is the result of the function and *what the function sqrt "returns"'*.

Math.pow() Example

Say we need to compute "6^2 + 5"

```
double result = Math.pow(6,2) + 5;
System.out.println(result); // -> 41.0
```

What will the answer be? 279936.0 or 41.0?

How did Java solve it?

Well, 6^2 (6 squared) is the same as 6 * 6. And 6 * 6 = 36, then add 36 + 5 = 41.

You'll learn a lot more about working with numbers in your career as a coder. This is really just the very basic of beginnings.

6.5. Chapter Exercises

Try these exercises in jshell to practice arithmetic operators:

Exercise 1: Basic Arithmetic

Practice all the basic operators:

```
int a = 20;
int b = 6;

System.out.println("a + b = " + (a + b));
System.out.println("a - b = " + (a - b));
System.out.println("a * b = " + (a * b));
System.out.println("a / b = " + (a / b));
System.out.println("a % b = " + (a % b));
```

What happens with integer division? Why is 20/6 = 3?

Exercise 2: Order of Operations

Predict the results before running:

```java
int result1 = 5 + 3 * 2;
int result2 = (5 + 3) * 2;
int result3 = 10 - 4 / 2 + 1;
int result4 = (10 - 4) / (2 + 1);

System.out.println("5 + 3 * 2 = " + result1);
System.out.println("(5 + 3) * 2 = " + result2);
System.out.println("10 - 4 / 2 + 1 = " + result3);
System.out.println("(10 - 4) / (2 + 1) = " + result4);
```

Exercise 3: Math Class Methods

Experiment with Math class methods:

```java
double radius = 5.0;
double area = Math.PI * Math.pow(radius, 2);
double circumference = 2 * Math.PI * radius;

System.out.println("Circle with radius: " + radius);
System.out.println("Area: " + area);
System.out.println("Circumference: " + circumference);
System.out.println("Square root of 64: " + Math.sqrt(64));
System.out.println("4 to the power of 3: " + Math.pow(4, 3));
```

Exercise 4: Increment and Decrement

Practice with ++ and — operators:

```java
int counter = 10;
System.out.println("Initial counter: " + counter);

counter++;
System.out.println("After counter++: " + counter);

counter--;
System.out.println("After counter--: " + counter);

System.out.println("Using ++counter: " + (++counter));
System.out.println("Using counter++: " + (counter++));
```

```
System.out.println("Final counter: " + counter);
```

Notice the difference between counter and counter

[1] The answer to life, the universe and Everything.

Chapter 7. Algebraic Equations

Some of the most fundamental of computer programs have been ones that take the drudgery of doing math by a person, and making the computer do the math. These kinds of *computations* rely on the fact that the computer won't do the wrong thing if it's programed carefully.

Given a simple math equation like: **Problem: Solve for a**

a = b3 - 6 and if b equals 3, then a equals ?

In math class, your teacher would have said "How do we solve for a?" The best way to solve for `a = b3 - 6` is to

- figure out what b times 3 is (well, if b equals 3, then 3 times 3 is 9)
- subtract 6 from b times 3 (and then 9 minus 6 is 3)

```
// And in Java:
// a = b3 - 6

int b = 3;
int a = b * 3 - 6;
System.out.println(a); // -> 3
```

Now you try it.

Exercise: Solve the equation with Java

q = 2j + 20

if j = 5, then q = ?

Take a moment and write down your solution before reading on.

```
int q = 0;
```

```
int j = 5;
q = 2 * j + 20;
System.out.println(q); // -> 30
```

Let's try another...

Exercise: Solve the equation with Java

$x = 5y + y^3 - 7$

if y=2, x = ?

and print out x.

My solution is pretty simple.

```
double y = 2;
double x = 5 * y + Math.pow(y, 3) - 7;
System.out.println(x); // -> 11
```

7.1. *Trigonometry*

The word trigonometry comes from the Greek words, trigonon ("triangle") + metron ("measure"). We use trigonometry to find angles, distances and areas.

For example, if we wanted to find the area of a triangular piece of land, we could use the equation **AreaOfaTriangle = height * base / 2**

Therefore we just need to create variables for each and use the operators to calculate the area.

Exercise: Calculate Area of a Triangle in Java

Height is 20 Base is 10 Formula: A = h * b / 2

```
double height = 20;
double base = 10;
double areaOfaTriangle  =  height * base / 2;
System.out.println(areaOfaTriangle); // -> 100
```

Exercise: Calculate the area of a circle whose radius is 7.7

Formula: area = Pi * radius^2 Hint: You'll need to use a constant Math property!

```
double radius = 7.7;
double area  =  Math.PI * Math.pow(radius, 2);
System.out.println(area);
// -> 186.26502843133886 (wow)
```

7.2. Chapter Exercises

Try these exercises in jshell to practice algebraic equations and trigonometry:

Exercise 1: Linear Equations

Solve these equations with Java:

```
// Equation: y = 3x + 7, where x = 4
int x = 4;
int y = 3 * x + 7;
System.out.println("When x = " + x + ", y = " + y);

// Equation: temperature in Fahrenheit = (C * 9/5) + 32
double celsius = 25.0;
double fahrenheit = (celsius * 9.0 / 5.0) + 32;
System.out.println(celsius + "°C = " + fahrenheit + "°F");
```

Exercise 2: Quadratic Expressions

Calculate values for a quadratic equation:

```
// Equation: y = x² + 2x - 3
int x = 5;
double y = Math.pow(x, 2) + 2 * x - 3;
System.out.println("For x = " + x + ", y = " + y);

// Try with different x values
x = -1;
y = Math.pow(x, 2) + 2 * x - 3;
System.out.println("For x = " + x + ", y = " + y);
```

Exercise 3: Geometry Calculations

Calculate areas and perimeters:

```
// Rectangle
double length = 12.5;
double width = 8.0;
double rectArea = length * width;
double rectPerimeter = 2 * (length + width);

System.out.println("Rectangle: " + length + " x " + width);
System.out.println("Area: " + rectArea);
System.out.println("Perimeter: " + rectPerimeter);

// Circle
double radius = 6.5;
double circleArea = Math.PI * Math.pow(radius, 2);
double circumference = 2 * Math.PI * radius;

System.out.println("Circle with radius: " + radius);
System.out.println("Area: " + circleArea);
System.out.println("Circumference: " + circumference);
```

Exercise 4: Physics Formulas

Calculate distance, speed, and acceleration:

```java
// Distance = speed x time
double speed = 65.0; // mph
double time = 2.5;   // hours
double distance = speed * time;
System.out.println("Driving " + speed + " mph for " + time + "
hours");
System.out.println("Distance traveled: " + distance + " miles");

// Force = mass x acceleration
double mass = 10.0;  // kg
double acceleration = 9.8; // m/s²
double force = mass * acceleration;
System.out.println("Force = " + mass + " kg x " + acceleration +
" m/s² = " + force + " N");
```

Chapter 8. Simple Calculation Programs

8.1. How far can we go in the car?

Let's create a simple problem to solve with Java.

> Our car's gas tank can hold 12.0 gallons of gas. It gets 22.0 miles per gallon (mpg) when driving at 55.0 miles per hour (mph). If we start with the tank full and carefully drive at 55.0 mph, how many miles can we drive (total miles driven) using the whole tank of gas?
>
> BONUS:
>
> How long will it take us to drive all those miles?

What do we need figure out? We need a variable for our result: totalMilesDriven. So we start our program this way...

```
double totalMilesDriven = 0;

// print result of computation
System.out.println(totalMilesDriven);
```

It's often good to start with a very simple template. If we run that, we will see 0 (zero) as the result, right?

Next step, let's add the variables we know.

```
double totalMilesDriven = 0;
double totalHoursTaken = 0;

double totalGasGallons = 12.0;
```

```
double milesPerGallon = 22.0;
double milesPerHour = 55.0;

// print result of computation
System.out.println(totalMilesDriven);
```

Okay, good. We’ve added all the stuff we know about the computation. Well, except the part of the *actual computation*.

You probably know that if you multiply the milesPerGallon by the totalGasGallons, that will give you totalMilesDriven. And if you divide the totalMilesDriven by the milesPerHour, you will get the totalHoursTaken.

So let’s add those as Java statements.

```
double totalMilesDriven = 0;
double totalHoursTaken = 0;

double totalGasGallons = 12.0;
double milesPerGallon = 22.0;
double milesPerHour = 55.0;

totalMilesDriven = milesPerGallon * totalGasGallons;
totalHoursTaken = totalMilesDriven / milesPerHour;

// print result of computation
System.out.println(totalMilesDriven +" "+ totalHoursTaken);
```

We get as a result 264 miles driven in 4.8 hours. And that’s how a simple Java program can get written.

Let’s do another.

8.2. The Cost of a "Free" Cat

A friend of ours is offering you a "free cat". You’re not allergic to cats but before you say yes, you want to know how much it’ll cost

to feed the cat for a year (and then, approximately how much much each month).

> We find out that cat food costs $2 for 3 cans. Each can will feed the cat for 1 day. (Half the can in the morning, the rest in the evening.) We know there are 365 days in a year. We also know that there are 12 months in the year. So how much will it cost to feed the cat for a year?

Looking at it, this may be quite simple. If we know each can feeds the cat for a day, we then know that we need 365 cans of food. So we can describe that as

```
double totalCost = 0;
double cansNeeded = 365;
double costPerCan = 2.0 / 3.0;

totalCost = cansNeeded * costPerCan; // right?

double monthsPerYear = 12;
double costPerMonth = totalCost / monthsPerYear;
// print result of computation
System.out.println(totalCost +" "+ costPerMonth);
```

What's going to be the answer? [1] Run it in jshell to work it all out.

And let's do one more.

8.3. You Used Too Much Data!

A cell phone company charges a monthly rate of $12.95 and $1.00 a gigabyte of data. The bill for this month is $21.20. How many gigabytes of data did we use? Again, let's use a simple template to get started.

```
double dataUsed = 0.0;

// print result of computation
System.out.println("total data used (GB)" + dataUsed);
```

Let's add what we know: that the monthly base charge (for calls, and so on) is $12.95 and that data costs 1 dollar per gigabyte. We also know the monthly bill is $21.20. Let's get all that written down.

```
double dataUsed = 0.0;
double costPerGB = 1.0;
double monthlyRate = 12.95;

double thisBill = 21.20;

// print result of computation
System.out.println("total data used (GB)" + dataUsed);
```

Now we're ready to do the computation. If we subtract the monthlyRate from thisBill, we get the total cost of data. Then, if we divide the total cost of data by the cost per gigabyte, we will get the dataUsed.

```
double dataUsed = 0.0;
double costPerGB = 1.0;
double monthlyRate = 12.95;

double thisBill = 21.20;
double totalDataCost = thisBill - monthlyRate;

dataUsed = totalDataCost / costPerGB;

// print result of computation
System.out.println();
System.out.println("total data used (GB)" + dataUsed);
```

How many GBs of data did we use? Turns out to be 8.25 gigabytes.

Now if the bill was $24.00? How many GBs then? (go ahead, I'll wait...) [2]

[1] totalCost will be $243.33 and $20.28 per month.

[2] total data used (GB) 11.05

Chapter 9. Boolean Expressions

When starting out in programming, the idea of a boolean variable, something that is either just true or false, seems like an overly simple thing… something that feels rather useless.

In fact, booleans in computer code are **everywhere**. They are simple, but also useful in many ways. You've probably heard about how everything in computers is ones and zeros at the lowest level - and that's true. But on this super simple base of `0 and 1` is built all of the power of the internet, and all the apps you've ever used.

When you are coding, you often have to make a choice about what to do next based on some kind of condition or to do something repetitively (over and over) based on some condition. Something like `is there gas in the car?` or `are we moving faster than 100mph?` In real life, these are considered to be YES (or **true**) or NO (or **false**) kinds of questions. If the gas tank is empty, the question results in a FALSE condition. If there is some gas in the tank, then the question's result is TRUE, "yes, there is some gas in the tank."

And while this may seem super simple to you, and it is, it is also very powerful when used in a program.

This idea of a condition that is either TRUE or FALSE, is known as a **boolean expression**. And in Java, they crop up everywhere. They are in *conditional statements* and they are part of *loops*.

Boolean expressions can be very complex, or very simple:

```
playerOne.isAlive() == true
```

might be a key thing to know inside of a game. But it might be more complicated:

```
player[1].isAlive() == true && player[2].isAlive() == true &&
spaceStation.hasAir() == true
```

(here, *&&* means `and`)

All three things need to be true to continue the game. Using boolean expressions, we can build very powerful tests to make sure everything is just as we need it to be.

We also need more kinds of boolean expressions when we are programming. Things like **less than** or **greater than or equal to**, and other **comparison operators** so we can compare things to work out the relationships within our data.

9.1. Comparison Operators

```
int healthScore = 5;
```

We need a way to ask about expressions like "is healthScore less than 7?? (very healthy)" or "is healthScore greater than or equal to 3?? (maybe barely alive?)"

To do that we need a bunch of **comparison operators**.

Operator	Description	Example
==	Equal to	x == 5
!=	Not equal to	x != 55
>	Greater than	x > 1
<	Less than	x < 10
>=	Greater than or equal to	x >= 5

Operator	Description	Example
<=	Less than or equal to	x <= 5

Each of these can be used to make it very clear to someone reading your code what you meant. Imagine a flight simulator, where you're flying a big, old fashioned airplane. The code that keeps track of the status of the plane might need to be able to make decisions on boolean expressions like:

```
altitude > 500.0   // high enough to not hit any trees!
airspeed >= 85.0   // fast enough to stay in the air.

fuelAvailable <= 5.0   // need to land to refuel!

totalCargoWeight < 6.0  // more than 6 tons and we can't take
off!

pilot.isAlive() && copilot.isAlive()  // both are alive, keep
flying.
```

In Java, ` == ` is used for *equals* in numbers (both *int* and *double* AND *boolean*). But when we compare two Stings, we use `.equals()`, like this `player1name.equals("Roberto")`. In Java you almost **never** want to use `player1name == "Roberto"`, always use `.equals()` instead.

Like the && (and) in the examples above or this last boolean expression with the "&&" in it, we have in Java the ability to combine expressions into larger more complex expressions using `&&` (and) and `||` (or).

9.2. Logical Operators

The **logical operators** are AND and OR, except in Java we use **&&** for AND and || for OR.

Operator	Description	
&&	Logical AND	playerOneStatus == 'alive' && spacecraft.hasAir()
||	Logical OR	room.Temp > 70 || room.Temp < 75

The AND operator, '&&', is an operator where BOTH sides have to be true for the expression to be true.

```
(5 < 9) && (6-3 == 3)  // true
```

See how both expressions on either side of the '&&' are true? That makes the entire line true.

The OR operator, '||' [1], is an operator where if ONE or the OTHER or BOTH boolean expressions are true, the entire expression is true.

```
(5 < 9) || (6-3 == 3) // true

(5 == 4) || (7 > 3)   // true!

(5 == 4) || (6 == 2) // false (both are false)
```

Both sides of a logical operator need to be Boolean expressions. So it's all right to use lots of different comparisons, and combine them with && and ||.

```
// deep in a cash machine application...
(customer.balance() <= 20.00
&&
customer.hasOverDraftProtection() == true)
||
(customer.savings.balance() > 20.0
```

```
&&
customer.canTransferFromSavings() )
```

See how these conditions could line up to allow a customer to get cash from the cash machine? Again, this is **why** boolean expressions are important and powerful and why coders need to be able to use them to get the software **just right**.

Exercise: Boolean Comparisons

- Create 2 variables to use for a comparison
- Use at least two comparison operators in Java
- And print them "System.out.println(2 > 1)"

Here is an example:

```
double houseTemp = 67.0;
double thermostatSetting = 70.0;

System.out.println(houseTemp >= 55.0);
System.out.println(houseTemp <= thermostatSetting);
System.out.println(thermostatSetting != 72.0);
System.out.println(houseTemp > 65.0 && thermostatSetting ==
68.0);
```

These log statements should produce TRUE, TRUE, TRUE and FALSE.

9.3. Chapter Exercises

Try these exercises in jshell to practice boolean expressions and logical operators:

Exercise 1: Basic Comparisons

Practice comparison operators:

```
int age = 22;
int votingAge = 18;
int drinkingAge = 21;

System.out.println("Age: " + age);
System.out.println("Can vote: " + (age >= votingAge));
System.out.println("Can drink: " + (age >= drinkingAge));
System.out.println("Is exactly voting age: " + (age ==
votingAge));
System.out.println("Is not drinking age: " + (age !=
drinkingAge));
```

Exercise 2: Logical AND and OR

Combine conditions with && and ||:

```
int temperature = 75;
boolean isSunny = true;
boolean hasUmbrella = false;

boolean perfectWeather = (temperature > 70) && isSunny;
boolean needJacket = (temperature < 60) || (!isSunny);
boolean goOutside = perfectWeather || (hasUmbrella && !isSunny);

System.out.println("Temperature: " + temperature + "°F, Sunny: "
+ isSunny);
System.out.println("Perfect weather: " + perfectWeather);
System.out.println("Need jacket: " + needJacket);
System.out.println("Go outside: " + goOutside);
```

Exercise 3: String Comparisons

Practice with string equality:

```
String password = "secret123";
String userInput = "secret123";
String wrongInput = "Secret123";

boolean correctPassword = password.equals(userInput);
boolean incorrectPassword = password.equals(wrongInput);
```

```
boolean isEmpty = password.equals("");

System.out.println("Correct password: " + correctPassword);
System.out.println("Incorrect password: " + incorrectPassword);
System.out.println("Password is empty: " + isEmpty);
System.out.println("Password length > 5: " + (password.length() >
5));
```

Exercise 4: Complex Conditions

Create a grade evaluation system:

```
int score = 85;
boolean hasExtraCredit = true;
boolean attendedAllClasses = true;

boolean passingGrade = score >= 70;
boolean excellentGrade = score >= 90;
boolean bonusEligible = hasExtraCredit && attendedAllClasses;
boolean finalPass = passingGrade || (score >= 60 &&
bonusEligible);

System.out.println("Score: " + score);
System.out.println("Passing grade: " + passingGrade);
System.out.println("Excellent grade: " + excellentGrade);
System.out.println("Bonus eligible: " + bonusEligible);
System.out.println("Final pass: " + finalPass);
```

[1] shift-backslash '\' on most keyboards

Chapter 10. Strings

Strings are perhaps the most important data type in Java. Many other computer languages have strings, and they are used in almost ALL modern programs. Knowing how to manage them, create and modify them to do what you need them to do, is a "sub-superpower" within Java.

Pay close attention; this stuff is VERY important.

10.1. What is a String?

Think about the words on this page. The text here is made up of a bunch of letters, and spaces. Now, when we write by hand, we don't really think about the space between the words, do we? If we truly ignored the notion of space between the words, wewouldendupwithtextlikethis. And while it is possible to read, our modern eyes are trained on well-edited texts; having no spaces tires us pretty quickly.

So yes, what we see as text in this book is really a series of letters and spaces strung together in a line - line after line, paragraph after paragraph. In modern computing, that kind of data is often called a **String**. It is one of the most fundamental aspects of coding: the manipulation of strings by programs to transform, present or store text in some fashion.

Many programming languages use some kind of quote or double quote to show where strings start and end. There is really no difference between using single or double quotes in Java. So a string like "the quick brown fox" would be a string from the 't' to the 'x'. And notice the three spaces within the string. If they were not there, the string would be "thequickbrownfox". And that's important, because to the computer, if it keeps these two strings around, it doesn't really understand that 'the' and 'quick' are just two common English words. The spaces are there to retain more

of what the human meant.

No, to the computer, each letter, including the space 'letter', is just a piece of data and very important.

String - a string of letters and numbers and spaces and punctuation, kept altogether for some use. Here are some strings for you to consider.

```
"the quick brown fox"
"The New York Times"
"And lo, like wave was he..."
"oops"
"Hello, World!"
"supercalifraglisticexpealadocious"
"On sale for $123.99!!"
"Pi is approximately 3.14159"
"Merge left at the ramp to the right, the restaurant is on the
right"
"He said, \"Wait there is more!\""
```

Think of strings as a tightly packed list. Each item and letter is numbered. The entire string can be "indexed", meaning I can reach in and copy out, say, the fifth letter easily. String indexes are zero-based; therefore, the first character (element) is in index position 0.

```
Index  -> 012345
String -> Hello
```

So here, "H" is at 0, "e" is at 1, 'l' is at 2 & 3, and 'o' is at index position 4. Computers often start numbering things like strings, lists, and arrays at 0, not at one. It’s just one of those things: all strings and arrays (which are coming up) start at zero.

10.2. Declaring a string

```
String name = "Wacka Flocka";
```

Now we have a string variable named **name** and it's value is "Wacka Flocka".

10.3. String Properties

A common and often used string property is **length()**.

We can use `.length` to find the length of a string

```
String str = "Wakanda Forever!";
int answer = str.length();
System.out.println(answer); // -> 16
```

10.4. Accessing Characters in a String

As mentioned before, we can reach into a string and copy out the stuff we find there.

```
String word = "Hello";
// Access the the first character (first by index, second by
function)
System.out.println(word.charAt(0)); // H
// the last character
System.out.println(word.charAt(word.length() - 1)); // o
```

When you see something like **word[0]**, it is pronounced like "word sub zero". If you have **word[5]**, you would say "word sub five". This is just verbal shorthand for the expression.

10.5. String Concatenation (Joining strings)

This simply means joining strings together using the + operator or the concat( ) method. Either one is commonly used.

```
int price = 20;
String dollarSign = "$";
String priceTag = dollarSign + price; // $20
//or
String priceTag = dollarSign.concat(String.valueOf(price)); //
$20
System.out.println(priceTag); // -> $20
```

Or perhaps a little more useful example:

```
String name = "Mikaila";
int hoursWorked = 12;

String workReport = "Today, " + name + " worked a total of " +
hoursWorked + " hours."
System.out.println(workReport);
```

The output would be:

```
Today, Mikaila worked a total of 12 hours.
```

10.6. SubStrings

Getting a substring is a common operation. This is how we extract the characters from a string, between two specified indices. (Which is why it's important to remember the indexes start at 0.)

*someString.**substring**(start, end)*

A start position is required, where to begin the extraction. Remember, first character is at position 0. Characters are extracted from a string between "start" and "end", not including "end" itself.

```
String firstName = "Christopher";
```

Now let's use the 3 substring methods on firstName and extract and print out "Chris"

```
String firstName = "Christopher";
System.out.println(firstName.substring(0,5)); // "Chris"
```

Yep. They all print "Chris". (Act impressed... thanks!) BUT, let's try to extract the string "stop" from the name.

```
String firstName = "Christopher";
System.out.println(firstName.substring(4,8)); // "stop"
```

Notice how the arguments to the functions are **slightly** different. This is why it might be best to pick to memorize and use that one.

Let's try a little harder idea...

Exercise: Extract and Transform Substring

```
String fName = "Christopher";
```

- Your turn to use the substring/substr/slice method on firstName
- Extract and print out "STOP" from inside the string above
- And make it uppercase! ("stop" to "STOP") [1]

Well?

```
String fName = "Christopher";
System.out.println(fName.substring(4,8).toUpperCase());
```

Want to bet there is also a "toLowerCase()" method as well?

10.7. Summary of substring methods

Take a look at these various ways to copy out a substring from the source string named 'rapper', which contains the string 'mikaila'.

```
String rapper = "mikaila";

System.out.println(rapper.substring(0,4));  // mika
System.out.println(rapper.substring(1,4));  // ika

System.out.println(rapper.substring(4,7));  // ila
System.out.println(rapper.substring(3,7));  // aila
```

We're using each of the three different substring methods to copy out some smaller piece of the 'rapper' string.

10.8. Chapter Exercises

Try these exercises in jshell to practice working with strings:

Exercise 1: String Basics

Practice string creation and basic properties:

```
String firstName = "Alice";
String lastName = "Johnson";
String fullName = firstName + " " + lastName;

System.out.println("First name: " + firstName);
```

```
System.out.println("Last name: " + lastName);
System.out.println("Full name: " + fullName);
System.out.println("Full name length: " + fullName.length());
System.out.println("First character: " + fullName.charAt(0));
System.out.println("Last character: " + fullName.charAt(fullName
.length() - 1));
```

Exercise 2: String Methods

Explore different string methods:

```
String message = "Java Programming is Fun!";

System.out.println("Original: " + message);
System.out.println("Uppercase: " + message.toUpperCase());
System.out.println("Lowercase: " + message.toLowerCase());
System.out.println("Length: " + message.length());
System.out.println("Contains 'Java': " + message.contains(
"Java"));
System.out.println("Contains 'Python': " + message.contains
("Python"));
System.out.println("Starts with 'Java': " + message.startsWith
("Java"));
System.out.println("Ends with '!': " + message.endsWith("!"));
```

Exercise 3: Substring Practice

Extract parts of strings:

```
String email = "alice.johnson@example.com";

int atIndex = email.indexOf("@");
String username = email.substring(0, atIndex);
String domain = email.substring(atIndex + 1);

System.out.println("Email: " + email);
System.out.println("Username: " + username);
System.out.println("Domain: " + domain);

// Extract first and last name from username
int dotIndex = username.indexOf(".");
```

```java
String first = username.substring(0, dotIndex);
String last = username.substring(dotIndex + 1);

System.out.println("First name: " + first);
System.out.println("Last name: " + last);
```

Exercise 4: String Building

Create formatted output:

```java
String productName = "Laptop";
double price = 1299.99;
int quantity = 2;
double total = price * quantity;

String receipt = "Product: " + productName + "\n" +
                "Price: $" + price + "\n" +
                "Quantity: " + quantity + "\n" +
                "Total: $" + total;

System.out.println("=== RECEIPT ===");
System.out.println(receipt);
System.out.println("===============");

// Practice with StringBuilder for better performance
StringBuilder sb = new StringBuilder();
sb.append("Order Summary:\n");
sb.append("Item: ").append(productName).append("\n");
sb.append("Cost: $").append(total);
System.out.println(sb.toString());
```

[1] You could google how to do this, try "Java string make upper case"

Chapter 11. Arrays

Arrays are a very powerful idea in many programming languages. Let's start with **why** we need them.

Imagine you have a small number of things you want to track. Let's use our vague computer game we've been using for an example. The game has 5 players, friends that get together over the internet to play a dungeon game.

Now, if you're the coder of this game you could keep track of each player's healthScore by have 5 different variables. (for players Zero to Four)

```
int playerZeroHealthScore = 100;
int playerOneHealthScore = 100;
int playerTwoHealthScore = 100;
int playerThreeHealthScore = 100;
int playerFourHealthScore = 100;
```

If we setup these 5 variables, our game can track 5 players! But **we'd have to change the game's code to track SIX players.** Well, that's not good. Kind of silly actually.

To get around this kind of problem we use an **array**. We could ask, "how many players are playing?", and then make an array that size. We know we need to track each player's healthScore, so we create an array:

```
int[] playerHealthScores = new int[]{100, 100, 100, 100, 100};
```

Now, like a *string*, array indexes start at zero.

```
                                //   0    1    2    3    4
```

```
int[] playerHealthScores = new int[]{100, 100, 100, 100, 100};
```

This array is a **data structure** - a way for us to keep track of lots of data in a controlled fashion. (We can make arrays any size we'd like.) If we need to deduct health points from one of the players, we can do something like this:

```
int majorHit = 50;

playerHealthScores[2] = 67;  // player 2 just took a hit!

playerHealthScores[1] = 105;  // player one is getting stronger.

playerHealthScores[3] = playerHealthScores[3] - majorHit;
```

The best way to think about arrays is like all those postal boxes at the post office. Each box has a number on it, and things get put in the box depending on the box number.

Arrays are **indexed** like that. Each array slot has an index number, starting at zero. See how we use the number 2 to *index* into the playerHealthScore array?

Arrays:

- Can store multiple values in a single variable
- Start counting from index position zero
- Elements can be primitive data types or/and Objects

So let's think about an array of donuts for the following examples.

11.1. Declaring Arrays

Declaring and initializing some arrays in Java:

```
String[] donuts = new String[]{"chocolate", "glazed", "jelly"};
```

11.2. Accessing elements of an Array

We use square brackets to get elements by their index. We'll use an array of strings to identify our donuts. Sometimes, we say something like "donuts sub 2" to mean *donuts[2]*.

```
String[] donuts = new String[]{"chocolate", "glazed", "jelly"};

System.out.println(donuts[0]);  // "chocolate" (we could say
"donuts sub zero")

System.out.println(donuts[2]);  // "jelly"
```

11.3. Get the size of an Array

We can use the **length** property to find the size of an array.

```
String[] donuts = new String[]{"chocolate", "glazed", "jelly"};

System.out.println(donuts.length);  // it'll print 3
```

Note: A string is kind of like ARRAY of single characters.

11.4. Get the last element of an Array

If we use the *length* property carefully, we can always get the last element in an array.

```
String[] donuts = new String[]{"chocolate", "glazed", "jelly",
"strawberry"};
```

```
System.out.println(donuts[donuts.length - 1]); // strawberry
```

11.5. Change an element of an Array

We can change the value within an array to anything we'd like, as long as it is the same **type** as the array started with.

```
System.out.println(donuts[2]); // jelly

                          // .    0            1         2
3
donuts[2] = "powdered";   // -> ["chocolate", "glazed",
"powdered", "strawberry"]

System.out.println(donuts[2]); // powdered
```

11.6. Chapter Exercises

Try these exercises in jshell to practice working with arrays:

Exercise 1: Array Basics

Create and manipulate a simple array:

```
int[] scores = {95, 87, 92, 78, 88};

System.out.println("First score: " + scores[0]);
System.out.println("Last score: " + scores[scores.length - 1]);
System.out.println("Array length: " + scores.length);

// Change a score
scores[1] = 90;
System.out.println("Updated second score: " + scores[1]);

// Print all scores
System.out.println("All scores:");
for (int i = 0; i < scores.length; i++) {
    System.out.println("Score " + (i + 1) + ": " + scores[i]);
```

```
}
```

Exercise 2: String Arrays

Work with an array of strings:

```
String[] cities = {"New York", "Los Angeles", "Chicago",
"Houston", "Phoenix"};

System.out.println("Cities in our array:");
for (int i = 0; i < cities.length; i++) {
    System.out.println((i + 1) + ". " + cities[i]);
}

// Find the longest city name
String longest = cities[0];
for (int i = 1; i < cities.length; i++) {
    if (cities[i].length() > longest.length()) {
        longest = cities[i];
    }
}
System.out.println("Longest city name: " + longest);
```

Exercise 3: Array Calculations

Calculate statistics from an array of numbers:

```
double[] prices = {12.99, 8.50, 15.75, 22.00, 9.99, 18.25};

// Find sum and average
double sum = 0;
for (int i = 0; i < prices.length; i++) {
    sum += prices[i];
}
double average = sum / prices.length;

// Find min and max
double min = prices[0];
double max = prices[0];
for (int i = 1; i < prices.length; i++) {
    if (prices[i] < min) min = prices[i];
```

```
    if (prices[i] > max) max = prices[i];
}

System.out.println("Number of items: " + prices.length);
System.out.println("Total cost: $" + sum);
System.out.println("Average price: $" + average);
System.out.println("Cheapest item: $" + min);
System.out.println("Most expensive item: $" + max);
```

Exercise 4: Enhanced For Loop

Try the enhanced for loop (for-each):

```
String[] fruits = {"apple", "banana", "orange", "grape", "kiwi"};

// Traditional for loop
System.out.println("Traditional for loop:");
for (int i = 0; i < fruits.length; i++) {
    System.out.println("- " + fruits[i]);
}

// Enhanced for loop (for-each)
System.out.println("\nEnhanced for loop:");
for (String fruit : fruits) {
    System.out.println("- " + fruit.toUpperCase());
}

// Count fruits that start with vowels
int vowelCount = 0;
for (String fruit : fruits) {
    char firstChar = fruit.charAt(0);
    if (firstChar == 'a' || firstChar == 'e' || firstChar == 'i'
||
        firstChar == 'o' || firstChar == 'u') {
        vowelCount++;
    }
}
System.out.println("Fruits starting with vowels: " + vowelCount);
```

Chapter 12. Changing the Control Flow

In many of these examples so far, we see a very simple **control flow**. The program starts at the first line, and just goes line by line until runs out of statements.

```
int q = 0;
int j = 5;
q = j * 4 - 20;
System.out.println(q); // -> 0
```

When programs start to get more sophisticated, the *control flow* can be changed. There are various conditional statements, loop statements, and functions that can cause the control flow to move around within the code. Here we see using both a loop and a conditional IF statement to change the flow of control.

```
int q = 0;
int j = 6;
while (j > 0) {
    q = j * 4 - 20;
    System.out.println(q);
    j--;
    if (q > 0) {
        System.out.println("q is still positive");
    }
}
```

This ability to manipulate the control flow of a program is very important when you start developing logic for your apps and programs. Logic in programs depends heavily on being able to manipulate the control flows through the code. Let's take a look at how each kind of statement allows a programmer to change the flow of control in programs.

Chapter 13. Conditional Statements

We have been seeing programs which consist of a list of statements, one after another, where the "flow of control" goes from one line to the next, top to bottom, and so on to the end of the list of lines. There are more useful ways of breaking up the "control flow" of a program. Java has several conditional statements that let the programmer do things based on conditions in the data.

13.1. If statement

The first conditional statement is the **if** statement.

```
if (something-is-true) {
    doSomething;
}
```

Here are a few simple examples.

```
if (speed > speedLimit) {
    driver.getsATicket();
}

if (x <= -1) {
    System.out.println("Cannot have negative numbers!");
}

if (account.balance >= amountRequested) {
    subtract(account, amountRequested);
    produceCash(amountRequested);
    printReceipt(amountRequested);
}
```

Java also has an **else** part to the **if** statement. When the **if**

condition is False, the else part gets run. Here, if the account doesn't have enough money to fulfill the amountRequested, the else part of the statement gets run, and the customer gets an insufficient funds receipt.

```
if (account.balance >= amountRequested) {
    // let customer have money
} else {
    printReceipt("Sorry, you don't have enough money in your
account!")
}
```

Java can also "nest" if statements, making them very flexible for complicated situations. You can also see here how curly-braces make it clear what statements get executed based on which case or condition is true.

```
String timeOfDay = "Afternoon";

if (timeOfDay.equals("Morning")) {
    System.out.println("Time to eat breakfast");
    eatCereal();
} else if (timeOfDay.equals("Afternoon")) {
    System.out.println("Time to eat lunch");
    haveASandwich();
} else {
    System.out.println("Time to eat dinner");
    makeDinner();
    eatDinner();
    doDishes();
}
```

Notice how this becomes a 3-way choice, depending on the timeOfDay.

Exercise: Age Verification

Write code to check if a user is old enough to drink.

- if the user's age is under 18. Print out "Cannot party with us"
- Else if the user's age is 18 or over, Print out "Party over here"
- Else print out, "I do not recognize your age" You should use an if statement for your solution!

Finally, make sure to change the value of the age variable in jshell, to output different results and test that all three options can happen. What do you have to do to make the `else` clause happen?

```
int userAge = 17;
if (userAge < 18) {
    System.out.println("Cannot party with us");
} else if (userAge >= 18) {
    System.out.println("Party over here");
} else {
    System.out.println("I do not recognize your age");
}
```

If statements are one of the most commonly used statements to express logic in a Java program. It's important to know them well.

13.2. Chapter Exercises

Try these exercises in jshell to practice if statements and conditional logic:

Exercise 1: Simple If Statements

Create a temperature checking program:

```
int temperature = 75;

if (temperature > 80) {
    System.out.println("It's hot outside!");
}

if (temperature < 60) {
```

```
    System.out.println("It's cold outside!");
}

if (temperature >= 60 && temperature <= 80) {
    System.out.println("Perfect weather!");
}

System.out.println("Current temperature: " + temperature + "°F");
```

Try changing the temperature value to test different conditions.

Exercise 2: If-Else Statements

Create a grade calculator:

```
int score = 85;
String grade;

if (score >= 90) {
    grade = "A";
} else if (score >= 80) {
    grade = "B";
} else if (score >= 70) {
    grade = "C";
} else if (score >= 60) {
    grade = "D";
} else {
    grade = "F";
}

System.out.println("Score: " + score);
System.out.println("Grade: " + grade);

if (score >= 70) {
    System.out.println("Passing grade!");
} else {
    System.out.println("Need to retake the test.");
}
```

Exercise 3: Nested If Statements

Create a movie recommendation system:

```
int age = 16;
String movieType = "action";
boolean hasParent = false;

System.out.println("Age: " + age + ", Movie type: " + movieType);

if (age >= 18) {
    System.out.println("Can watch any movie");
} else {
    if (movieType.equals("kids")) {
        System.out.println("Can watch kids movies");
    } else if (movieType.equals("teen") && age >= 13) {
        System.out.println("Can watch teen movies");
    } else if (hasParent) {
        System.out.println("Can watch with parent supervision");
    } else {
        System.out.println("Cannot watch this movie");
    }
}
```

Exercise 4: Complex Conditions

Create a login validation system:

```
String username = "admin";
String password = "secret123";
boolean accountLocked = false;
int loginAttempts = 2;

if (accountLocked) {
    System.out.println("Account is locked. Contact
administrator.");
} else if (loginAttempts >= 3) {
    System.out.println("Too many failed attempts. Account
locked.");
} else if (username.equals("admin") && password.equals
("secret123")) {
    System.out.println("Login successful! Welcome, " + username);
} else {
    System.out.println("Invalid username or password.");
```

```
    System.out.println("Attempts remaining: " + (3 -
loginAttempts - 1));
}
```

13.3. Switch Statement

Switch statements are used to perform different actions based on different conditions.

Here we have a long example, where the user types a command, and the program runs the code for a given command. (This is sometimes referred to as a shell or command loop).

Notice how we first get a command from the user, look at each possibility, and do something specific if we find a matching command string. Otherwise, we print an error message.

```
String lastCommand = getCommandFromUser();

switch(lastCommand){
    case "exit":
        System.out.println("so long!");
        break;
    case "run":
        System.out.println("running simulation...");
        runSim();
        break;
    case "rename":
        renameSim();
        break;
    case "delete":
        if (makeSureDeleteIsOkay()) {
            deleteSim();
        } else {
           System.out.println("delete cancelled...");
        }
        break;
    case "new":
        createNewSim();
        break;
    case "help":
```

```
        showHelpToUser();
        break;
    default:
        System.out.println("command not found: try again or type
help");
}
```

Switch statements can be quite elaborate (and in this case much better than a whole lot of IF statements).

Here's the exercise from the *if* section. Oftentimes, you can write a switch statement in *if* statements, or a bunch of *if* statements as a *switch* statement. This time, you should use a **switch** statement.

Exercise: Age Verification with Switch Statement

Write code to check if a user is old enough to drink. (Using a switch)

- if the user is under 18. Print out "cannot party with us"
- Else if the user is 18 or over. Print out "party over here"
- Else print out "I do not recognize your age"

Finally, make sure to change the value of the age variable to output different results.

Chapter 14. Loops

Loops allow you control over repetitive steps you need to do in your *control flow*. Java has two different kinds of loops we will talk about: **while** loops and **for** loops. Either one can be used interchangeably; but, as you will see there are couple cases where using one over the other makes more sense.

The primary purpose of loops is to avoid having lots of repetitive code.

14.1. While Loop

Loop through a block of code (the body) WHILE a condition is true.

```
while (condition_is_true) {

    // execute the code statements
    // in the loop body

}
```

See the code below. In this case, we start with a simple counter in x = 1. Then, after the loop starts, it checks to see if x < 6, and 1 is less than 6, so the loop body gets executed. We print out 1 and then increment x. Then we go to the top of the loop and check to see if x (now 2) is less than 6. Since that's true so we print out 2 and increment x again. This continues like this for three more times, printing 3, 4, and 5.

Then, x is incremented to 6, and the check is made again, 6 < 6 … well, no that is false. So we don't execute the loop's body and we fall through to the last System.out.println line, and print out x.

```
int x = 1;
```

```
while (x < 6) {
    System.out.println(x);
    x++;
}

System.out.println("ending at " + x); // ? what will print here ?
```

While loops work well in situations where the condition you are testing at the top of the loop is one that may not be related to a simple number.

```
while (player[1].isAlive() == true) {
    player[1].takeTurn();
    game.updateStatus(player[1]);
}
```

This will keep letting player[1] take a turn in the game until the player dies. Another way to do something like this is with an *infinite loop*. (No, infinite loops are not necessarily a bad thing, watch.) We're going to use both **continue** and **break** in this example, and we will describe them better after we're done with loops.

```
Game game = new Game();
Player player = game.newPlayer();

while (true) { // <- notice right here, an infinite loop

    player.takeTurn();
    game.updateScores();
    game.advanceTime();

    if (player.isAlive() == true) {
        continue; // start at top of loop again.
    } else {
        break; // breaks out of loop and ends game.
    }
}
```

```
game.sayToHuman("Game Over!");
```

Here, we are using the continue statement to force the flow of control to the top of the loop. We are also using the break statement to break out of the infinite loop, letting us do other things after the player has 'died'.

14.2. Do..While Loop

There is another kind of loop, a **Do..While** loop. Why? well, sometimes you need a loop to go at least once, no matter what and continue until the condition on the loop becomes false. These are used only occasionally, and only in very specific situations.

```
int x = 0;

do {
    System.out.println(x);
    x++;
}
while (x < 5);
```

14.3. For Loop

The **for** loop is more complex, but it's also the most commonly used loop.

```
for (begin; condition; step) {

    // execute the code statements
    // in the loop body

}
```

Here's one where we go from 1 to 5.

```
for(int j = 1; j < 6;  j++){
    // loop body code
    System.out.println(j);
}
```

This loop will print out:

```
// print
1
2
3
4
5
```

Notice how there are THREE parts to the FOR loop's mechanism.

- begin: j = 1 // Executes once upon entering the loop.
- condition: j < 6 // Checked before every loop iteration. If false, the loop stops.
- loop step: j++ // Executes after the body on each iteration.

and

- body: System.out.println() // Runs again and again while the condition is true.

Let's show you another glimpse of the **break** statement.

```
for(int p = 1; p < 6; p++){
    if(p == 4){
        break;
    }
System.out.println("Loop " + p + " times");
}
```

Jumps out of the loop when p is equal to 4.

Exercise: Basic Loop Practice

- Print from 10 to 1 with a for loop and a while loop (hint use decrement)
- Write a loop that prints 1 - 5 but break out at 3

Stretch Goal: S/he who dares wins!

- Go back to Arrays
- Look at an array of donuts
- Create an array of donuts
- Loop through the array of donuts and print each donut string

You can do it, I know you can!

```
for(int x = 0; x < donuts.length; x++){
    System.out.println(donuts[x]);
}
```

If you had something like this, buy yourself a donut, you deserve it.

14.4. Chapter Exercises

Try these exercises in jshell to practice for loops:

Exercise 1: Basic For Loop

Practice counting and basic output:

```
// Count from 1 to 10
for (int i = 1; i <= 10; i++) {
    System.out.println("Count: " + i);
}

// Count backwards from 10 to 1
```

```
System.out.println("\nCountdown:");
for (int i = 10; i >= 1; i--) {
    System.out.println(i);
}
System.out.println("Blast off!");

// Count by 2s
System.out.println("\nEven numbers from 2 to 20:");
for (int i = 2; i <= 20; i += 2) {
    System.out.println(i);
}
```

Exercise 2: Mathematical Calculations

Use for loops for calculations:

```
// Calculate sum of numbers 1 to 100
int sum = 0;
for (int i = 1; i <= 100; i++) {
    sum += i;
}
System.out.println("Sum of 1 to 100: " + sum);

// Create a multiplication table
int number = 7;
System.out.println("\nMultiplication table for " + number + ":");
for (int i = 1; i <= 10; i++) {
    System.out.println(number + " x " + i + " = " + (number *
i));
}

// Calculate factorial
int factorial = 1;
int n = 5;
for (int i = 1; i <= n; i++) {
    factorial *= i;
}
System.out.println(n + "! = " + factorial);
```

Exercise 3: Pattern Printing

Create visual patterns with loops:

```java
// Print a triangle of stars
System.out.println("Triangle:");
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}

// Print a square
System.out.println("\nSquare:");
for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= 4; j++) {
        System.out.print("# ");
    }
    System.out.println();
}

// Print numbers in a pattern
System.out.println("\nNumber pattern:");
for (int i = 1; i <= 4; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(j + " ");
    }
    System.out.println();
}
```

Exercise 4: Working with Arrays

Combine for loops with arrays:

```java
String[] names = {"Alice", "Bob", "Charlie", "Diana", "Eve"};

// Print all names with index
System.out.println("Student roster:");
for (int i = 0; i < names.length; i++) {
    System.out.println((i + 1) + ". " + names[i]);
}

// Find names that start with a specific letter
char searchLetter = 'A';
System.out.println("\nNames starting with '" + searchLetter +
```

```
":");
for (int i = 0; i < names.length; i++) {
    if (names[i].charAt(0) == searchLetter) {
        System.out.println("- " + names[i]);
    }
}

// Create a modified version
System.out.println("\nFormatted names:");
for (int i = 0; i < names.length; i++) {
    System.out.println("Hello, " + names[i] + "!");
}
```

14.5. Break Statement

Normally, a loop exits when its condition becomes false. But we can force the exit at any time using the special **break** statement.

```
while (true) {

  String cmd = getCommandFromUser("Enter a command", "");

  if (cmd.equals("exit")) break;

  execute(cmd);
}
System.out.println("Exiting.");
```

Here, you are asking the user to type in a command. If the command is "exit", then quit the loop and output "Exiting", and end the program. Otherwise, execute the command and go around to the top of the loop and ask for another command.

14.6. Continue Statement

The continue statement doesn't stop the whole loop. Instead, it stops the current iteration and forces the loop to start a new one (if the condition allows).

We can use it if we're done with the current iteration and would like to move on to the next one. This loops prints odd number less than 10.

```
for (int i = 0; i < 10; i++) {

  // if true, skip the remaining part of the body
  // will only be true if the number is even

  if (i % 2 == 0) continue;

  System.out.println(i); // prints 1, then 3, 5, 7, 9
}
```

What's interesting here is the use of the remainder operator (%) to see if a number is odd. The expression (i % 2) is zero if the number is even, if not, the number must be odd. You want to remember this trick of how to find odd or even numbers. It's a common programming problem that you will get asked. The continue statement starts the loop over, not letting the System.out.println() to print out the number when it's even.

Chapter 15. Code Patterns

Any experienced coder would say that the ability to see patterns in code, remember them, and learn from them when creating code is another kind of 'superpower'. The following samples are really simple techniques, but they show some common ways of doing things that you should think about and study. In almost all these examples, there may be some missing variable declarations. Just roll with it. If you think about it, I'm sure you can figure out what variable declarations are needed to run the sample in jshell.

15.1. Simple Patterns

If you wanted to find the larger of two values, x and y and assign it to 'max':

```
if (x > y) {
    max = x;
} else {
    max = y;
}
```

Related to it, if we have two variables x and y, and we want the smaller in x, and the larger in y.

```
if (x > y) {
    int t = x;
    x = y;
    y = t;
}
```

Do you see the three statements in the block there? That's called a 'swap'. If you need to swap two values in two variables, you just create a quick temporary variable 't' and use it as a place to make a copy of the first variable's value.

If I needed to make sure a number is always positive (greater than zero), it's easy - this is called taking the "absolute value" of a number.

```
if (n < 0) n = -n;
```

15.2. Loop Patterns

The next few are examples of the handy use of loops to do a bunch of math easily and quickly. Imagine a problem where you have to "add all the numbers from 1 to 100 and print the sum." It might also be expressed as "**sum** all the number from x to y" (where x and y are two integers). Turns out there is a very easy pattern to learn here.

```
int sum = 0;
int n = 100;
for (int i = 1; i < n; i++) {
    sum = sum + i;
}
System.out.println(sum);
```

Now, if you wanted to find the average of a bunch of numbers, that's as easy as taking the sum of the numbers and dividing the sum by the number of numbers (or n).

```
int sum = 0;  // this needs to be 0 not 1
int n = 100;
for (int i = 1; i < n; i++) {
    sum = sum + i;
}
int average = sum / n;
System.out.println(average);
```

Pretty easy, yes? And the other common pattern here is doing a

product of all the numbers from 1 to n. (Let's try 20)

```
int product = 1; // this needs to be 1 not 0
int n = 20;
for (int i = 1; i < n; i++) {
    product = product * i;
}
System.out.println(product);
```

Perhaps you want to print a table of values of some equation.

```
for (int i = 0; i <= n; i++) {
    System.out.println(i + " " + i*i/2);
}
```

15.3. Array Patterns

Arrays are often something that confuses beginning coders. Let's look at some code patterns with arrays that let you see how arrays and loops can work together to get a lot of work pretty easily.

The array we are going to use in all these cases is pretty simple. It's an array of 7 numbers.

```
int[] a = {4, 3, 7, 0, -4, 1, 8};
```

Here how to print out the array, one value per line.

```
for (int i = 0; i < a.length; i++) {
    System.out.println(i + " " + a[i]); // print the index and
value of an array element.
}
```

If we needed to find the **smallest** number in the array, we could

do:

```
int min = a[0];
for (int i = 1; i < a.length; i++) {
    if (a[i] < min) min = a[i];
}
System.out.println(min);
```

We should look carefully here. First, notice how I have taken the first element a[0] and made my first 'min' that value. Then, I started at 1 (not 0), to be my first compare. Then we step through the array, looking at each value and if the new value is smaller than the previous one, we update it; otherwise, we just do the next value. [1]

NOW, if you wanted to find the **largest** value in the array, you really only have to change a couple things.

```
int max = a[0];
for (int i = 1; i < a.length; i++) {
    if (a[i] > max) max = a[i];
}
System.out.println(max);
```

Carefully look at the code, comparing to the one above. What's different? Well, for one, we changed the variable from 'min' to 'max'. (But did we need to do that? We could have left it max, but it's cleaner to make the change so people who read it aren't confused.) We also changed the comparison in the 'if' statement from "less than <" to "greater than >" which lets us decide if the new number is larger than the previous largest we found.

In both of these cases, we start with an initial value, then we step through the array, look at each value comparing it to the smallest (or largest) value we have yet found. If we need to update the 'carrying variable', we do; otherwise, we just ignore the value.

What about finding the average of the values in the array? Well, we do it a lot like the average of the series of numbers.

```
int sum = 0;
for (int i = 0; i < a.length; i++) {
    sum = sum + a[i];
}
int average = sum / a.length ; // whoa! lookee there?

System.out.println(average);
```

Yep, the "a.length" is very handy, it has exactly the count of the numbers in the array!

Finally, if we wanted to reverse the values in the array, we could write some code:

```
System.out.println("before: " + Arrays.toString(a));
int n = a.length;
int half = (int) Math.ceil(n / 2.0);
for (int i = 0; i < half; i++) {
    int t = a[i];
    a[i] = a[n-1-i];
    a[n-i-1] = t;
}
System.out.println("after: " + Arrays.toString(a));
```

[1] YES, if the array is only one element long, this will fail. But I'm merely trying to show some concepts here. I'd do this differently, if it were to be in some codebase somewhere.

Chapter 16. Classes

In Java, modules allow for code to loaded into a program only if it is needed. Modules are one of the advanced topics in Java that we won't spend too much time on, but here are the basics.

Most of your Java programs are fairly small when you are creating solutions to HackerRank type problems. They can be seen as a simple script you store in a file on your computer. Usually, when they are stored on your computer, they are ".js" files.

```
System.out.println("Hello, World!");
```

helloworld.js

If your program is much larger, it might be split into different files to keep it all more organized or readable. All those files might be kept in a folder all together, as a project. But again, this is beyond what you need to know to do HackerRank Java problems.

Modules are also used to import code others have written that you wish to take advantage of. There are millions of chunks of Java you can find and use in your code. A lot of it is used by many, many people, and it's important to know where the code you use comes from. It can be dangerous to use someone else's code that isn't trustworthy.

See https://Java.info/modules-intro for more on modules! Look for information on "import" and "export" to see how modules interact with your code.

Chapter 17. Methods

A method is a block of code designed to perform a particular task. A method encloses a set of statements and is a fundamental modular unit of Java. They let you reuse code, and provide a way for you to organize your programs, keeping them easier to understand and easier to modify. It's often said that the craft of programming is the creation of a set of methods and data structures which implement a solution to some problem or set of requirements.

methods are objects, like many things in Java. They can be stored in variables, other objects, or even collected into arrays. methods are very powerful because in Java, they can be passed as arguments to other methods, returned from methods, and can even have methods attached to them. The most important thing that methods can do is get **invoked**.

You create methods easily.

17.1. method Literal

Two examples of how you write a method literal:

```
public static int add(int parameter1, int parameter2) {
    return parameter1 + parameter2;
}
```

```
public static int add(int p1, int p2) {
    return p1 + p2;
}
```

In both of these cases, you end up with a method named 'add' that takes two parameters and adds them, returning the result. (Yes, you could just write (p1 + p2) and everyone would understand.

It's just a very simple example.) Here is another example:

17.2. Creating a method

```
public static void greetUser(String username) {
    System.out.println( "Hello " + username);
}

//calling/Invoking the method
greetUser("Mike Jones"); // "Hello Mike Jones"
```

17.3. Invoking methods

methods are meant to be *invoked*. So you can have one method call another method which calls a third method and so on; it's very common.

Imagine we have a program that gets an airplane ready for flight. We can imagine a whole series of methods we'd have to write. Things like 'loadPassengers', 'loadBaggage', 'loadFuel', 'checkTirePressures', and so on. Then we might have a 'higher level' methods which brings all these pieces together:

```
public static void prepFlight(Airplane airplane) {
    loadBaggage(airplane);
    loadPassengers(airplane);

    loadFuel(airplane);
    performPreflightChecklist(airplane.copilot);
    askTowerToDepart(airplane.pilot);
    departGate(airplane);

    taxiToRunway(airplane, mainRunway90);
    takeoff(airplane.pilot, airplane);
}
```

You can see how methods let you perform different things in a

particular order, and while you might have no idea *how* 'loadBaggage' does what it does, you can see how the program preps the airplane object for flight and makes sure everything important is done. Each of the methods from 'loadBaggage' to 'takeoff' is invoked and returns so the next one can be invoked. This is the power of methods - the ability to take some code and put it a method so that it much easier to understand.

17.4. Anonymous methods

A common pattern used in Java revolves around an **anonymous** method. It's a method without a name, and one that probably only gets called *once*. Something like:

```
// Anonymous methods are not typically used in Java like this
// Here's a simple method example instead:
public static void exampleMethod() {
    int x = 5;
    int y = 4;
    while (x < 100) {
        x = x + y;
    }
    System.out.println("x is " + x);
}
```

Look at that last line. What's going on here? Well, first we are creating a method object *without a name* (so it's *anonymous*). Then on the last line, we are *invoking* the method once (see those two parentheses? '()'?), so all the code inside gets run. (x get printed out).

You see this often in Java. It's a way to put some code into an argument to another method call.

```
int milliSecInASecond = 1000;

// Note: setTimeout doesn't exist in standard Java
```

```
// This is more like JavaScript syntax
// In Java, you would use Timer or ScheduledExecutorService
System.out.println("We waited for 5 seconds, to print this");
```

See how we pass an anonymous method to 'setTimeout' as one argument, and a timer value as the second argument? The idea here is that the anonymous method gets called at 5 seconds in the future. (Like scheduling an alarm to do something in 5 seconds).

17.5. method Return

Once Java reaches a return statement, the method will stop executing. methods often compute a return value. The return value is "returned" back to the "caller". You can have many returns in a methods, depending on how the flow of control is changed.

```
public static String greetUser(String username) {
    return "Hello " + username;
}

String result = greetUser("Welcome back, Mike Jones");

System.out.println(result); // will print "Hello Welcome back,
Mike Jones"
```

Or like this:

```
public static String determineWinner(Team home, Team visitor) {
    if (home.score > visitor.score) {
        return "Home Team Wins! Let's have a Parade!";
    } else if (home.score < visitor.score) {
        return "Visitors Win! (oh Well)";
    }
    return "It's a Tie!";
}
```

Notice how in this case, we check to see the scoring results with two conditions (which are, what? yes, *boolean expressions*). If neither condition is true, the third one must be the case. But if either condition is true, then we return right away, and the method is done.

Again, to be clear, we might use this method like this:

```
Team home = new Team();
home.name = "Fightin Cats";
home.score = 0;

Team visitor = new Team();
visitor.name = "Wild Horses";
visitor.score = 0;

playGame(home, visitor); // a lot of work done in this method(!)

// game is done
String result = determineWinner(home, visitor);

// and then print the result..
System.out.println(result);
```

17.6. method Parameters

methods can also take parameters to be used within a method.

```
public static int addThreeNumbers(int a, int b, int c) {
    return (a + b + c);
}

public static String determineWinner(Team home, Team visitor) {
    if (home.score > visitor.score) {
        return "Home Team Wins! Let's have a Parade!";
    } else if (home.score < visitor.score) {
        return "Visitors Win! (oh Well)";
    }
    return "It's a Tie!";
}
```

```
public static int makeNegative(int number) {
    if (number > 0) {
        return -(number);
    }
    // already negative, it's less than 0
    return number;
}
```

Remember how we had the expression to see if a number was even? (x % 2 == 0) Now, here's a way to decide is number was divisible cleanly by another, it's a standard arithmetic expression:

```
(number % divisor == 0)
```

So to see if a number is even, we could use '(number % 2 == 0)':

```
System.out.println((8 % 2 == 0)); // true
System.out.println((7 % 2 == 0)); // false
System.out.println((4 % 2 == 0)); // true
```

And we can use the same technique to see if a number is evenly divisible by 3 or 5.

Try to write a method that will perform the following requirements:

Exercise: Create a ZipCoder Method

- Create a method called zipCoder
- Your method takes one parameter of type number
- Your method checks and does the following
- If parameter is divisible by 3 and 5 (15). Print ZipCoder
- If parameter is divisible by 3. Print Zip

- If parameter is divisible by 5. Print Coder Phew...Finally
- Call the method and pass in 45 as your parameter

OKAY! Write it yourself!

Do it.

Just write it yourself.

C'mon, write your own version first.

No, really.

Wait.

Do you want to be a ZipCoder, or just a Copy-Paste Stylist?

Well, here's one solution:

```
// method ZipCoder

public static void zipCoder(int aNumber) {
    if (aNumber % 15 == 0) System.out.println("ZipCoder");
    else if (aNumber % 3 == 0) System.out.println("Zip");
    else if (aNumber % 5 == 0) System.out.println("Coder");
}

zipCoder(45); // -> ZipCoder
```

Chapter 18. Return statement

The **return** statement is a very simple one. It just finishes the running of code in the current method and "returns" to the method's caller.

A method in Java can return a value if the method needs to.

As you have seen, *methods* are used to make code more understandable, cleaner and more organized. Say we have a couple of functions in our program:

```
int minorHit = 3;
int majorHit = 7;

public boolean adjustHealth(Player player, int hit) {
    player.health = player.health - hit;

    return isAlive(player);
}

public boolean isAlive(Player player) {
    if (player.health >= 20) {
        return true;
    } else { // player has died!
        return false;
    }
}
```

If someplace in our code we were to do something like:

```
// big hit!
isPlayerDead = adjustHealth(playerOne, majorHit);

if (isPlayerDead == true) endGame();
```

You can see how when we call the function "adjustHealth()" it returns the result of calling `isAlive(player)`, and we make a

decision to end the game if the player has died.

Notice too, you can have multiple return statements in functions, and each one can return a different value if that's what you need.

If a method is marked `void` then the method does not return a value, but can still use `return` statements to stop computing within a method.

```
public void printStatus(String message) {
    System.err.println(message);
    return;
}
```

In this `void` return case the return statement does not return a value; but it makes it clear the method is at an end.

18.1. Chapter Exercises

Try these exercises in jshell to practice working with return statements and methods:

Exercise 1: Simple Return Method

Create a method that takes two integers and returns their sum:

```
int addNumbers(int a, int b) {
    return a + b;
}
```

Test it: `addNumbers(5, 3)` should return `8`

Exercise 2: Boolean Return Method

Create a method that checks if a number is positive:

```
boolean isPositive(int number) {
    return number > 0;
}
```

Test it: `isPositive(5)` should return `true`, `isPositive(-3)` should return `false`

Exercise 3: String Return Method

Create a method that returns a greeting message:

```
String createGreeting(String name) {
    return "Hello, " + name + "!";
}
```

Test it: `createGreeting("Alice")` should return `"Hello, Alice!"`

Exercise 4: Multiple Return Paths

Create a method that returns different messages based on a grade:

```
String getGradeMessage(int grade) {
    if (grade >= 90) {
        return "Excellent!";
    } else if (grade >= 70) {
        return "Good job!";
    } else {
        return "Keep trying!";
    }
}
```

Test it with different values: `getGradeMessage(95)`, `getGradeMessage(75)`, `getGradeMessage(65)`

Chapter 19. Objects

Java is a language which encourages the idea of Object Oriented Programming. **Objects** are a fundamental idea in Java. In a Java program, a programmer will break down the problem into smaller pieces, each piece might be a small grouping of objects.

Let's think about our reliable furnace, the one that keeps us warm all winter long. Attached to the furnace is a *thermostat*, a very simple computer. There is a 3-way switch on most of them, one that allows you to select *off*, *heat* or *cool*. In the winter, of course, if you're in the northern hemisphere, you tend to choose *heat*.

If the thermostat is rather old fashion, it might have a ring you rotate to set the temperature, say 68 degrees.

A Java programmer writes code which in *classes* and then, when a java program runs, *objects* are created All but one of them are called "primitive" data types, because their values contain only a single thing (be it a string or a number or whatever).

In contrast, **objects** are used to store **keyed** collections of various data and more complex entities. *Objects* add tremendous power to Java and penetrate almost every aspect of the language. Here are a few different objects, all these are actually objects in Java, which makes the language rather elegant:

- arrays
- functions
- regular expressions
- objects [1]

Objects are used to collect and organize data - and that data can be variable values, functions and other things. Objects can also contain other objects(!), in kind of a "nesting" way. This allows for large data structures to be built using a very simple and elegant

mechanism.

An object (in this case an *object literal*) can be created with curly braces {...} with an optional list of properties. A property is a “key: value” pair, where key is a string (also called a “property name”), and value can be anything. [2]

```
// Creating a Person object using a class
class Person {
    String firstname;
    String lastname;
    int homedistrict;
    String[] skills;

    public Person(String firstname, String lastname, int
homedistrict, String[] skills) {
        this.firstname = firstname;
        this.lastname = lastname;
        this.homedistrict = homedistrict;
        this.skills = skills;
    }
}

Person katniss = new Person("Katniss", "Everdene", 12,
    new String[]{"foraging", "wildlife", "hunting", "survival"});
```

Notice how you use commas for all but the last one in the list?

19.1. Object Creation

We can imagine an object as a container where everything has a key, and we can get data from the key, add data with a key to an object, and delete a key (with its data value) from the object. Each pair (of key and value) can be referred to as a **property**. Properties can be almost any Java data type at all. Let’s create a sample player object for a game:

```
// Creating a Player class with nested Weapons class
class Weapons {
```

```
    String sword;
    String bow;
    int arrows;

    public Weapons(String sword, String bow, int arrows) {
        this.sword = sword;
        this.bow = bow;
        this.arrows = arrows;
    }
}

class Player {
    String name;
    String species;
    String superPower;
    int lives;
    Weapons weapons;

    public Player(String name, String species, String superPower,
int lives, Weapons weapons) {
        this.name = name;
        this.species = species;
        this.superPower = superPower;
        this.lives = lives;
        this.weapons = weapons;
    }
}

Player player1 = new Player("Rocco", "warrior", "flight", 4,
    new Weapons("Excalibur", "Legolas", 12));
```

A property has a key (also known as “name” or “identifier”) before the colon ":" and a value to the right of it.

Here 'name' is a *key* and "Rocco" is the *value*. This 'player1' object has 5 key/value pairs (and you can add as many as you like, and can delete those you don’t need). One of those key/value pairs is a nested object, the 'weapons' key has another object with 3 key/value pairs inside of it. (Notice the commas ',' after all but the last item in each object.)

19.2. Property Retrieval

Values can be retrieved from inside an object by using a string expression in a [ ] (bracket) pair. You can also use "dot notation".

```
player1.name   // "Rocco"
player1.lives  // 4
```

You get 'undefined' if you try to retrieve a value whose key doesn't exist.

19.3. Property Update

If we were to use an assignment statement, we can change the properties inside of the object.

```
player1.name = "Gina";
player1.species = "Mage";
player1.weapons.sword = "Anduril";  // notice how to reach into
nested objects!

System.out.println(player1.weapons.sword);  // Anduril
```

19.4. Object Reference

This next little bit will stretch your mind a bit.

If we have an object:

```
class Airplane {
    String type;
    double topSpeed;
    int passengerLimit;
    int passengersOnboard;
    String destination;
    String origin;
```

```
    public Airplane(String type, double topSpeed, int
passengerLimit, int passengersOnboard) {
        this.type = type;
        this.topSpeed = topSpeed;
        this.passengerLimit = passengerLimit;
        this.passengersOnboard = passengersOnboard;
    }
}

Airplane airplane512 = new Airplane("airbusA330", 0.86, 253, 0);
```

We have built a plane object.

```
Airplane flight77 = airplane512;  // I'm assigning a plane to a
flight

flight77.destination = "Tokyo"; //create two new properties in
the object
flight77.origin = "Seattle";

// now watch carefully...
flight77.passengersOnboard = 120;

// and if I print out some of the properties...
System.out.println(airplane512.passengersOnboard);  // 120
System.out.println(flight77.type); // airbusA330
System.out.println(airplane512.type); // airbusA330
```

This is because both flight77 and airplane512 _refer to the **same** object. so when you make changes through the flight77 name, the properties in the object change, and you can get and/or set properties through the airplane512 name as well. This proves very powerful in many cases.

19.5. Object Functions

Now to make things even more powerful, Java objects can have *functions* attached to them. Let's build a special counter object,

something we might use in a program or app somewhere.

As you may be aware, in American football there are four common changes to a team's score during a game:

```
import java.util.HashMap;
import java.util.Map;

class ScoreCounter {
    private Map<String, Integer> options;
    private int score;

    public ScoreCounter() {
        options = new HashMap<>();
        options.put("fieldgoal", 3);
        options.put("touchdown", 6);
        options.put("point_after_touchdown", 1);
        options.put("safety", 2);
        score = 0;
    }

    public void touchdown() {
        this.score += this.options.get("touchdown");
    }

    public void fieldgoal() {
        this.score += this.options.get("fieldgoal");
    }

    public void pat() {
        this.score += this.options.get("point_after_touchdown");
    }

    public void safety() {
        this.score += this.options.get("safety");
    }

    public int getScore() {
        return this.score;
    }
}

ScoreCounter scoreCounter = new ScoreCounter();
```

We can use that object, with its function methods like this:

```
scoreCounter.touchdown(); // add 6
scoreCounter.pat(); // add 1
scoreCounter.fieldgoal(); // add 3

System.out.println(scoreCounter.getScore()); // ??
```

There are two data properties (options and score), and 5(!) functions. These functions are called **methods** (functions which are attached to an object), and get invoked when you make the *method call* (or *invoke* the method on the *object*).

Notice the **this** variable. *This* is the special variable used to refer to the *object itself*. (which is a rather advanced topic for this book, so, we'll leave it right there. When you get a chance, read about *this* in a deeper Java resource.)

19.6. Follow Ons

We have tried to give you some of the very basic parts of Java, in order for you to be able to do well on the Zip Code Wilmington assessment. (Or for you to get a very basic understanding of coding in Java and whether or not you enjoy learning this sort of thing.)

There are a number of very powerful things we have left out of this discussion about Java objects. We have not covered the ideas of **prototypes** and the **prototype chain** here, which are not really needed for the assessment you may be taking.

We also have not discussed an extremely powerful concept, **closures**. But rest assured, there is much much more for you to learn about Objects in Java.

Master what we've written about here and then forge ahead into more complicated and powerful capabilities.

There is a lot more to learn about Java. But you made it this far, so perhaps you have what it takes to learn the 21st century superpower of coding.

[1] Now, unlike a lot of languages, Java has no notion of *classes*. It uses a different model of *prototypes*.

[2] In some languages, a listing of key/value pairs is called a dictionary, an associative array or a hashtable.

Appendix A: Kris' Special Class

In this book, we use a simple template for all your early Java programs. Why? Well, in many books, tutorials, etcetera for Java, they use a very simple (well, for Java, simple) *Hello World* program.

blah, blah, blah

But in this book, I use a special template. One that you should use when working thru the examples and exercises in this book.

A.1. The SPECIAL ZipCode java template!

```
public class ZipCode {

  void compute() {

    //Your code goes in here...

  }


  // don't change this...
  public static void main(String args { new ZipCode().compute();
}
}

// NOTICE the } in the previous line? it's crucial
```

Appendix B: Advanced Ideas

We're going to look at a few "modern" ways of handling a collection of data. Frequently, you have a list, or an array, of data that needs to be gone through to print it out, transform it in some way, or to summarize it (such as a total or an average). As you have seen in the code patterns section, there are common loops used for such things, a simple pattern that you can memorize.

There are other methods of doing these things, and we're going to discuss a few of them here. These ideas are based primarily on methods made popular by Hadoop and other "big data" applications and tools. And what's good for "big" data is often good for "small" data as well.

Each of these sections is an example of a more "elegant" way of expressing coding logic. By studying each one and comparing it to the ways we've discussed before using loops and conditional statements, we're expanding your understanding, making you see how these techniques can be used to create more extensible and elegant code.

Let's use this array for the following examples.

```
const groceries = [
  {
    name: 'Breakfast Cereal',
    price: 5.50,
  },
  {
    name: 'Rice',
    price: 14.99,
  },
  {
    name: 'Oranges',
    price: 6.49,
  },
  {
    name: 'Crackers',
```

```
    price: 4.79,
  },
  {
    name: "Potatoes",
    price: 3.99,
  },
];
```

A common grocery list, we have this as a list (or array) of objects (what's known as a key/value data structure).

B.1. Simplifying Loops

Now, if you wanted to print out each item's name in the grocery list to the console, you could do something like this:

```
int idx = 0;
while (idx < groceries.length) {
  System.out.println(groceries[idx].name);
  idx = idx + 1;
}
```

This is a very common code pattern in Java. It's also fraught with possible errors. It relies on the `idx` variable. If we forget or mess up the increment step at the end of the loop, we could be in trouble. Rather, how about this:

```
groceries.forEach((item) => {
  System.out.println(item.name);
});
```

`forEach` is a *higher-order function* that takes in another function as an argument and executes the provided function once for each element in the array. It is meant to simplify your code. By using forEach, we remove the extraneous code for tracking and accessing the array using an index, and focus on our logic:

printing out the name of each grocery item.

A higher-order function is a function that does at least one of the following:

- takes one or more functions as arguments (i.e. procedural parameters),
- returns a function as its result.

Let's look at a few other uses of higher-order functions. Say we wanted to get a list of just the prices of our grocery list. If we use a loop, we have a very recognizable pattern.

```
int index = 0;
const prices = [];
while (index < groceries.length) {
  prices.push(groceries[index].price);
  index = index + 1;
}
```

But if we use a different higher-order function, `map`:

```
int prices = groceries.map((item) => {
  return item.price;
});
```

The value of prices would be: `[5.5,14.99,6.49,4.79,3.99]`. And if we were to want something a little more useful than just producing a list, we would do this when using a loop:

```
int index = 0;
int total = 0;
while (index < groceries.length) {
  total = total + groceries[index].price;
  index = index + 1;
}
```

Now we are tracking *two* pieces of information, the index and the total. With the sum of all the prices ending up in `total`.

But look how much simpler our code can be if we use `reduce`, another higher-order function.

```
int total = groceries.reduce((sum, item) => {
  return sum += item.price;
}, 0);
```

The result, if we print `total` is the awkward number `35.760000000000005`, and why that is, well, later we'll discuss it. But, `35.76` should suffice.

If we wanted to pull out all the even numbers from a list of numbers, we'd need a function like this to decide if a number if even. (Remember the trick using modulus?)

```
function even (value) {
    return value % 2 === 0;
}

System.out.println(even(3), even(4), even(126));

// giving us 'false true true'
```

If we remove the name `even` from the definition, and use a higher-order function named `filter`, we have something like this.

```
numList = [1,2,3,4,5,6,7,8];
numList.filter(
    function (value) {return value % 2 === 0}
);
```

`filter` is a function which filters out the elements in an array that don't pass a test. You can visualize it this way:

```
function filter(array, test) {
  let passed = [];
  for (let element of array) {
    if (test(element)) {
      passed.push(element);
    }
  }
  return passed;
}

function even (value) {
    return value % 2 === 0;
}

int dataList = [1,2,3,4];

System.out.println(
    filter(dataList, even)
); // produces [2,4]
```

See what I've done? First, I've shown you how to express `filter` with both a loop and an `if` statement, expressing the function in more verbose code to give you the idea of what's going on. Second, I've then used it to show it in action.

I've defined two functions, `filter` and `even`. Then created a short array/list called `dataList`. Finally, I've printed the result of calling `filter(dataList,even)`. Wait, what? I passed the function's name, `even` as an argument to another function. Well, sure, why not? Functions in Java are called `first class objects`, just like a variable or an object or a value.

And it turns out Java already has a function called `filter`, a higher order function. And I can reduce it even more to something like this:

```
int result = [1,2,3,4].filter(
    function (value) {return value % 2 === 0}
);
```

```
// or, If I have 'dataList' defined as [1,2,3,4]
int result = dataList.filter(
    function (value) {return value % 2 === 0}
);
```

And as you will see further down, we can even reduce that to a simpler form, called a `lambda`.

In these four cases, we see how we can use a different form of computing, a *functional* form, to simplify our code by removing loops and their trappings and replacing them with higher-order functions, letting us hand some of our logic to the language itself. And making our code more elegant in the process.

B.2. Simplifying Conditionals

We can use the same ideas with conditionals. Conditionals can get thick and complicated without too much effort. Say we need to keep track of and perform different discounts for various purposes. Sounds like an `if` statement! With `else` statements too!

But `else` statements, for instance, have a habit of complicating code.

Every time you add an else statement, you increase the complexity of your code two-fold. Conditional constructs like if-else and switch statements are foundational blocks in the world of programming. But they can also get in the way when you want to write clean, extensible code.

Let's create a function that computes a discount for a price amount based on sone discount code. We might, happily, build something like this:

```
const discount = (amount, code) => {
  if (code == 'TWENTYOFF') {
      return amount * 0.80;
```

```
  } else if (code == 'QUARTEROFF') {
      return amount * 0.75;
  } else if (code == 'HALFOFF') {
      return amount * 0.50;
  } else { // no discount
      return amount;
  }
}; // whew! that a lot of braces.

int netprice = discount(200.00, 'HALFOFF'); // would be 100.
```

But think about adding another discount, we'd have to add another `if`, more braces, and make sure we nest it in there carefully, otherwise we break the whole, rickety, mess.

I know! Let's use a `switch` statement, and simplify! Well...

Switch statements too, have a way of expanding on you, getting long, and sometimes complex, requiring care to maintain and/or extend. Say you wanted to add some more discounts to the following switch statement?

```
const discount = (amount, code) => {
  switch (code) {
    case 'TWENTYOFF':
      return amount * 0.80;
    case 'QUARTEROFF':
      return amount * 0.75;
    case 'HALFOFF':
      return amount * 0.50;
  }
};

int netprice = discount(200.00, 'HALFOFF'); // would be 100.
```

We have to add two lines of code for each `case`. And if you make a mistake, you break the whole contraption.

But consider this idea: use a combination of a simple data structure and a small piece of code (called an arrow function (or

"lambda")).

```
const DISCOUNT_MULTIPLIER = {
  'TWENTYOFF': 0.80,
  'QUARTEROFF': 0.75,
  'HALFOFF': 0.50,
};

const discount = (amount, code) => { // look at that arrow '=>'?
  return amount * DISCOUNT_MULTIPLIER[code];
};
```

Whoa! How easy is it to add another 1, 3 or 7 discount cases? Just one line each. This re-factor effectively decouples the data we use from the core calculation logic, which makes it much easier to modify either independently. No `ifs`, `elses` or `switches`, just an object holding data, a simple lambda (arrow) function which does simple math.

B.3. Lambdas (or Arrow Functions)

One of the ways we do a lot of this kind of simplification within code is by replacing more complex logic with simpler forms.

In Java, we have function expressions which give us an anonymous function (a function without a name). Here we are creating an anonymous function and assigning it to a variable.

```
var anon = function (a, b) { return a + b };

// this is the same as
function anon (a, b) {
    return a + b;
}
```

It's really just a different form of the same thing.

But we also have `lambdas` or `arrow functions` with a more flexible syntax that has some bonus features and gotchas. We could write the above example as:

```
var anon = function (a, b) { return a + b }; // from above

var anon = (a, b) => a + b; // Sweet!

// or we could
var anon = (a, b) => { return a + b };
// if we only have one parameter we can loose the parentheses
var anon = a => a + a;
// and without parameters
var () => {} // this does nothing. So who cares?

// this looks pretty nice when you change something like:
[1,2,3,4].filter(
    function (value) {return value % 2 === 0}
);
// to:
[1,2,3,4].filter(value => value % 2 === 0);
```

See how much easier it is to read the last line in the example over the previous filter using the anonymous function? Lambdas are a powerful way to express small functions, and use them in a variety of ways. They are often paired with higher-order functions, as they simplify the code quite a bit.

B.4. Polymorphism and K.I.S.S.

Remember "keep it simple, stupid"? Yeah, we suffer from over-complicating things in coding as well. Another way to replace conditionals is by using a key feature of object-oriented programming languages: polymorphism. Let’s show some code which helps bill a customer.

```
// list of customers we want to 'checkout'
const customers = [
```

```
  {
    name: 'sam',
    amount: 75.00,
    paymentMethod: 'credit-card',
  },
  {
    name: 'frodo',
    amount: 50.00,
    paymentMethod: 'debit-card',
  },
  {
    name: 'galadriel',
    amount: 25.00,
    paymentMethod: 'cash',
  },
];
```

I'm going to gloss over the code needed to do each of the three kinds of payment. But show you how I might have to account for all three inside a `checkout` function.

```
const checkout = (amount, paymentMethod) => {
  switch (paymentMethod) {
    case 'credit-card':
      // Complex code to charge ${amount} to the credit card.
      break;
    case 'debit-card':
      // Complex code to charge ${amount} to the debit card.
      break;
    case 'cash':
      // Complex code to put ${amount} into the cash drawer.
      break;
  }
};
```

Now, I'd like to take the list of customers, and checkout each one. (Notice how I'm using the higher-order function here, not a `for` loop.)

```
customers.forEach(({ amount, paymentMethod }) => {
```

```
  checkout(amount, paymentMethod);
});
```

But if I use polymorphism, I can make each customer's checkout method wired directly to the data list. And look how I have broken the large function up, into three simpler things.

```
class CreditCardCheckout {
  static charge(amount) {
    // Complex code to charge ${amount} to the credit card.
  }
}
class DebitCardCheckout {
  static charge(amount) {
    // Complex code to charge ${amount} to the debit card.
  }
}
class CashCheckout {
  static charge(amount) {
    // Complex code to put ${amount} into the cash drawer.
  }
}
const customers = [
  {
    name: 'sam',
    amount: 75.00,
    paymentMethod: CreditCardCheckout,
  },
  {
    name: 'frodo',
    amount: 50.00,
    paymentMethod: DebitCardCheckout,
  },
  {
    name: 'galadriel',
    amount: 25.00,
    paymentMethod: CashCheckout,
  },
];
customers.forEach(({ amount, paymentMethod}) => {
  paymentMethod.charge(amount);
});
```

I am using a `class` in this example, well, three of them actually. One for each payment method. I can put the complex code within each class, and if I set them all up to have a `charge` method (a method being the term we use to talk about a function wired to a class), I know I just need to call `charge` on each customer, and the classes will all figure out which piece of code to use. This is an example of polymorphism, "many forms, same name".

Another example, commonly used in explaining polymorphism, is a series of geometric shapes, like Square, Triangle and Circle. Each of those shapes has a different way of computing the `area` of itself. A Square's `area()` is `(side * side)`, right? But a Circle's `area()` is `(Math.PI * (radius * radius))`. Two different ways of calculating the area of a shape, depending on the kind of shape we're working with. Each of these shapes would have it's own class, each with a different definition of how to find the area of the shape. That's polymorphism in a nutshell.

Each of these techniques are currently considered "advanced" Java, even though in many cases they are simpler and less error-prone than more "traditional" loops and conditionals.

Be sure to consider how each of them are largely the same in functionality but simpler in expressing the logic of your program. Remember to make your code more elegant by adding more simplicity.

Appendix C: Mars Lander

This is some code to show you how you might write a simple mars lander simulation in Java. It's taken from history, way back in the 1970's - this idea was passed around as some of the very first open source programs.

Meant as an example of a longer program (159 lines) to get you thinking, it's really not very complicated. The general idea is you have a series of "burns" in a list, and the game (or simulator, if you will) steps through the list applying each burn. If you run out of altitude (or height) while you're going too fast, you will crash.

The tricky bit would be for you to figure out what `burnArray` would be used to safely land at a vehicle speed 1 or 2. That could be hard.

```
// Mars Lander Source Code.

const Gravity = 100
/* The rate in which the spaceship descents in free fall (in ten
seconds) */

const version = "1.2"; /* The Version of the program */

// various end-of-game messages.
const dead = "\nThere were no survivors.\n\n";
const crashed = "\nThe Spaceship crashed. Good luck getting back
home.\n\n";
const success = "\nYou made it! Good job!\n\n";
const emptyfuel = "\nThere is no fuel left. You're floating
around like Wheatley.\n\n";

function randomheight() {
    let max = 20000;
    let min = 10000;
    let r = Math.floor(Math.random() * (max - min)) + min;
    return (r % 15000 + 4000)
}

function gameHeader() {
```

```
    s = "";
    s = s + "\nMars Lander - Version " + version + "\n";
    s = s + "This is a computer simulation of an Apollo mars
landing capsule.\n";
    s = s + "The on-board computer has failed so you have to land
the capsule manually.\n";
    s = s + "Set burn rate of retro rockets to any value between
0 (free fall) and 200\n";
    s = s + "(maximum burn) kilo per second. Set burn rate every
10 seconds.\n"; /* That's why we have to go with 10 second-steps.
*/
    s = s + "You must land at a speed of 2 or 1. Good Luck!\n\n";
    return s;
}

function getHeader() {
    s = "";
    s = s + "\nTime\t";
    s = s + "Speed\t\t";
    s = s + "Fuel\t\t";
    s = s + "Height\t\t";
    s = s + "Burn\n";
    s = s + "----\t";
    s = s + "-----\t\t";
    s = s + "----\t\t";
    s = s + "------\t\t";
    s = s + "----\n";
    return s;
}


function computeDeltaV(vehicle) {
    return (vehicle.Speed + Gravity - vehicle.Burn)
}

function checkStatus(vehicle) {
    s = "";
    if (vehicle.Height <= 0) {
        if (vehicle.Speed > 10) {
            s = dead;
        }
        if (vehicle.Speed < 10 && vehicle.Speed > 3) {
            s = crashed;
        }

        if (vehicle.Speed < 3) {
```

```
            s = success;
        }
    } else {
        if (vehicle.Height > 0) {
            s = emptyfuel;
        }
    }
    return s
}

function adjustForBurn(vehicle) {
    // save previousHeight
    vehicle.PrevHeight = vehicle.Height;
    // compute new velocity
    vehicle.Speed = computeDeltaV(vehicle);
    // compute new height of vehicle
    vehicle.Height = vehicle.Height - vehicle.Speed;
    // subtract fuel used from tank
    vehicle.Fuel = vehicle.Fuel - vehicle.Burn;
}

function stillFlying() {
    return (vehicle.Height > 0);
}

function outOfFuel(vehicle) {
    return (vehicle.Fuel <= 0);
}

function getStatus(vehicle) {
    // create a string with the vehicle status on it.
    let s = "";
    s = vehicle.Tensec + "0 \t\t" + vehicle.Speed + " \t\t" +
vehicle.Fuel + " \t\t" +
        vehicle.Height;
    return s
}

function printString(string) {
    // print long strings with new lines the them.
    let a = string.split(/\r?\n/);
    for (i = 0; i < a.length; i++) {
        System.out.println(a[i]);
    }
}
```

```
// this is initial vehicle setup
var vehicle = {
    Height: 8000,
    Speed: 1000,
    Fuel: 12000,
    Tensec: 0,
    Burn: 0,
    PrevHeight: 8000,
    Step: 1,
}

// main game loop
function runGame(burns) {
    let status = ""

    /* Set initial vehicle parameters */
    let h = randomheight()
    vehicle.Height = h;
    vehicle.PrevHeight = h;

    burnIdx = 0;

    printString(gameHeader());
    printString(getHeader());

    while (stillFlying() === true) {

        status = getStatus(vehicle);

        vehicle.Burn = burns[burnIdx];
        printString(status + "\t\t" + vehicle.Burn);

        adjustForBurn(vehicle);

        if (outOfFuel(vehicle) === true) {
            break;
        }
        vehicle.Tensec++;
        burnIdx++;


    }
    status = checkStatus(vehicle);
    printString(status);
}
```

```
// these are the series of burns made each 10 secs by the lander.
// change them to see if you can get the lander to make a soft
landing.
// burns are between 0 and 200. This burn array usually crashes.
const burnArray = [100, 100, 200, 200, 100, 100, 0, 0, 200, 100,
100, 0, 0, 0, 0];

runGame(burnArray);
```

Appendix D: Installing Java JDK

To use jshell and run the examples in this book, you need Java JDK version 9 or later installed on your computer. This appendix will help you check if you already have Java installed and guide you through installation if needed.

D.1. Checking Your Current Java Installation

First, let's see if you already have Java installed and what version it is.

D.1.1. Step 1: Open Your Terminal or Command Prompt

On Windows: - Press `Windows + R`, type `cmd`, and press Enter - Or search for "Command Prompt" in the Start menu

On macOS: - Press `Cmd + Space`, type `terminal`, and press Enter - Or go to Applications → Utilities → Terminal

On Linux: - Press `Ctrl + Alt + T` - Or search for "Terminal" in your applications

D.1.2. Step 2: Check Java Version

Type the following command and press Enter:

```
java -version
```

You might see output like this:

```
java version "17.0.2" 2022-01-18 LTS
```

```
Java(TM) SE Runtime Environment (build 17.0.2+8-LTS-86)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.2+8-LTS-86, mixed
mode, sharing)
```

If you see a version number 9 or higher (like 11, 17, 21), you're all set!

If you see an error like "java is not recognized" or "command not found", you need to install Java.

If you see a version number lower than 9 (like 1.8), you should update to a newer version.

D.1.3. Step 3: Check if You Have the JDK (Not Just JRE)

To use jshell, you specifically need the JDK (Java Development Kit), not just the JRE (Java Runtime Environment). Check by typing:

```
javac -version
```

You should see something like:

```
javac 17.0.2
```

If you get "javac is not recognized" or "command not found", you have the JRE but need the full JDK.

D.2. Installing Java JDK

If you need to install or update Java, here are the recommended approaches:

D.2.1. Option 1: Oracle JDK (Recommended for Beginners)

1. Go to https://www.oracle.com/java/technologies/downloads/
2. Download the latest LTS (Long Term Support) version for your operating system
3. Run the installer and follow the setup wizard
4. Restart your terminal/command prompt
5. Test the installation using the commands from Step 2 above

D.2.2. Option 2: OpenJDK (Free Alternative)

OpenJDK is a free, open-source implementation of Java:

For Windows: 1. Go to https://adoptium.net/ 2. Download the latest LTS version 3. Run the installer 4. Make sure "Set JAVA_HOME variable" is checked during installation

For macOS with Homebrew:

```
brew install openjdk
```

For Ubuntu/Debian Linux:

```
sudo apt update
sudo apt install openjdk-17-jdk
```

For Red Hat/CentOS/Fedora Linux:

```
sudo yum install java-17-openjdk-devel
```

D.2.3. Option 3: SDKMAN (For Advanced Users)

SDKMAN is a tool for managing multiple Java versions:

1. Install SDKMAN: https://sdkman.io/install
2. Install Java:

```
sdk install java 17.0.2-tem
```

D.3. Verifying Your Installation

After installation, open a new terminal/command prompt and run:

```
java -version
javac -version
jshell
```

The first two commands should show version 9 or higher. The third command should start jshell with a welcome message like:

```
|  Welcome to JShell -- Version 17.0.2
|  For an introduction type: /help intro

jshell>
```

Type `/exit` to quit jshell.

D.4. Troubleshooting Common Issues

D.4.1. "java is not recognized as an internal or external command"

On Windows: 1. The Java installer should have set the PATH automatically 2. If not, you may need to add Java to your PATH manually: - Find where Java is installed (usually `C:\Program Files\Java\jdk-XX\bin`) - Add this to your system PATH environment variable - Restart your command prompt

On macOS/Linux: 1. Java might be installed but not in your PATH 2. Try: `which java` to see if it's found 3. You may need to add Java to your PATH in your shell configuration file

D.4.2. Multiple Java Versions

If you have multiple Java versions installed:

On Windows: The last installed version usually becomes the default

On macOS: Use `java_home` to manage versions:

```
/usr/libexec/java_home -V
```

On Linux: Use `update-alternatives`:

```
sudo update-alternatives --config java
```

D.4.3. Permission Issues on macOS

If you get security warnings on macOS: 1. Go to System Preferences → Security & Privacy 2. Click "Allow" for Oracle or the Java installer 3. Try the installation again

D.5. Recommended Java Versions

For this book, we recommend: - **Java 17 LTS** - Current long-term support version (recommended) - **Java 21 LTS** - Latest long-term support version - **Java 11 LTS** - Older but still supported LTS version

Avoid: - Java 8 and earlier (no jshell support) - Non-LTS versions unless you have a specific need

D.6. Getting Help

If you're still having trouble:

1. **Check your Java vendor's documentation:**
   - Oracle: https://docs.oracle.com/en/java/
   - Adoptium: https://adoptium.net/support/
2. **University/School IT Support:** Many institutions provide Java installation help
3. **Online Communities:**
   - Stack Overflow: https://stackoverflow.com/questions/tagged/java
   - Reddit: r/learnjava
4. **Verify Installation:**
   - Make sure you downloaded the JDK, not just the JRE
   - Ensure you downloaded the correct version for your operating system (32-bit vs 64-bit)
   - Try restarting your computer after installation

Once you have Java JDK 9 or later installed and working, you're ready to start using jshell and following along with all the examples in this book!

Appendix E: Additional Java Resources

Here are a series of other resources to go on from this point. Dev.java is a really good one.

Some Java sites for you to explore:

- https://dev.java
  - https://dev.java/learn/getting-started-with-java/
  - https://dev.java/learn/java-language-basics/

If you're looking for more of a professional code tool, use an IDE like vscode: https://code.visualstudio.com (Many people use this these days.)